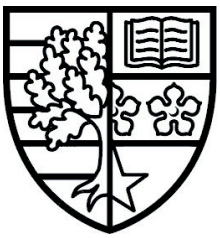
Interactive Gantt Chart Builder

by Fraser Davies

Research Report

Submitted in partial fulfilment for the degree of
BSc (Hons) Computer Science

Supervised by Dr Pierre Le Bras



Heriot-Watt University
School of Mathematical and Computer Sciences

9th April 2026

Authorship Declaration

Course code and name	F20PA - Honours Project
Type of assessment	Individual
Coursework title	Interactive Gantt Chart Builder
Student name	Fraser Davies
Student ID	H00403114

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.

- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the University's website, and that I am aware of the penalties that I will face should I not adhere to the University Regulations.

- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: Fraser Davies

Date: 9th April 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

Acknowledgements

I am profoundly grateful to express my sincere appreciation to all those who have contributed to the successful completion of this dissertation.

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr Pierre Le Bras, for his exceptional guidance and unwavering support throughout this research endeavour. His weekly meeting check-ups, insightful feedback, and constant encouragement have been instrumental in shaping and refining this work. Dr Le Bras's understanding nature, patience, and dedication to academic excellence have been a source of motivation, enabling me to navigate through the challenges encountered along the way.

I extend my heartfelt appreciation to Dr Kayvan Karim for setting aside his valuable time to support this project. His extraordinary patience and willingness to explain the intricacies of the backend architecture of the current Heriot-Watt University project system have been invaluable. Dr Karim's understanding approach, technical expertise, and thoughtful feedback have been crucial to the feasibility and implementation aspects of this research.

I would like to express my sincere thanks to Dr Richmond Davies for providing guidance and feedback on the overall dissertation. His helpful improvements and insightful contributions have significantly enhanced the quality of this work.

I am deeply grateful to Sarrah "Saly" Citroën for her invaluable assistance with travel arrangements between campus and home. Her help in managing timekeeping and coordinating transportation to the university campus has enabled me to focus on my research and maintain productivity throughout this demanding period.

Finally, I extend my appreciation to Heriot-Watt University and the School of Mathematical and Computer Sciences for providing the conducive learning environment and extensive resources that have been indispensable throughout this academic journey.

Thank you all for your support and belief in this project.

Abstract

Project management is critical for students in higher education, particularly those undertaking substantial individual projects such as honours dissertations. Despite the widespread adoption of Gantt charts as the fourth most used project management tool among professionals [8], their application in individual student dissertation contexts remains underexplored in academic literature. This dissertation investigates the design, development, and evaluation of an Interactive Gantt Chart Builder specifically designed for honours project students at Heriot-Watt University, with the primary objective of producing an interactive web-based tool that guides students through creating comprehensive project plans whilst offering supervisors visibility into student progress and planning quality.

The system was implemented as a standalone web application using Node.js with Express, SQLite, and D3.js, integrating interactive visualisation capabilities including drag-and-drop timeline editing, task dependencies with cycle detection, progress tracking, a Kanban board view, and supervisor view-only access. Requirements were gathered and prioritised using the MoSCoW method, identifying 22 functional and 7 non-functional requirements organised as user stories. Of the 22 functional requirements, all six Must Have requirements were fully implemented, nine of eleven Should Have requirements were fully implemented, and three of five Could Have requirements were delivered.

The system was evaluated through a fifteen-participant think-aloud user study with honours students, requirements-based system testing, and a stakeholder demonstration with the technical lead for the HWU project management system. The study produced a mean System Usability Scale score of 87.8, corresponding to an adjective rating of Excellent, well above the commonly cited average of 68. Qualitative feedback was particularly positive regarding the tutorial modal and the drag-and-drop interaction model, whilst subtask management and dependency creation were identified as areas for future improvement. The stakeholder assessment confirmed that the database structure is compatible with the existing HWU system, with integration work planned to begin in April 2026. These results demonstrate that an interactive, student-focused Gantt chart tool is both technically feasible and practically usable within the target academic context.

Table of Contents

1 Introduction

1.1 Background and Motivation

Project management is critical for students in higher education, particularly those undertaking substantial individual projects such as honours dissertations. Dissertations present unique challenges as they are often the largest and most complex projects students have encountered, requiring sustained effort over extended periods while juggling multiple responsibilities [46]. Effective planning, scheduling, and progress monitoring are essential skills that students must develop to successfully navigate these complex, long-term academic endeavours. However, many students struggle with translating abstract project requirements into concrete, manageable tasks with realistic timelines and dependencies, finding themselves overwhelmed by the scope and duration of dissertation work [46].

Gantt charts have emerged as one of the most widely adopted project management tools, originally developed by Henry Gantt in the early 20th century to visualise project schedules through horizontal bar representations [20, 49]. These visual tools effectively display task durations, dependencies, and timelines, making them invaluable for planning and tracking project progress [16]. In educational contexts, Gantt charts serve as pedagogical tools that help students manage their increasing responsibilities while developing time management skills. Their graphical representation visualises beginning dates, ending dates, and duration parameters, helping to track various projects over specific time periods [7].

Despite their proven utility in professional project management, traditional Gantt chart tools often present usability challenges for students who lack prior project management experience. Many existing solutions are designed for team-based projects and enterprise environments, making them overly complex for individual academic projects. Furthermore, the widespread adoption of visualisation tools in educational settings has been hindered by usability problems and the time required for instructors and students to learn and incorporate these tools into their workflows.

At Heriot-Watt University (HWU), honours project students are currently required to submit project management plans as part of their dissertation deliverables. However, the existing project management system (projects.hw.ac.uk)lacks integrated tools that guide students through the process of creating well-structured Gantt charts with appropriate task hierarchies, dependencies, and milestones. This gap in the current system motivated the development of an interactive Gantt chart builder specifically tailored to the needs of honours project students and their supervisors.

1.2 Aim and Objectives

The primary aim of this project is to design, develop, and evaluate an interactive Gantt chart builder that will be integrated into the existing HWU honours project management system.

This tool will provide structured guidance to help students create comprehensive project plans while offering supervisors visibility into student progress and planning quality.

The specific objectives of this project are:

(1) To conduct comprehensive background research on Gantt chart design principles, project management methodologies, and interactive visualisation techniques relevant to educational contexts
(2) To gather and analyse requirements from key stakeholders to ensure the tool meets user needs
(3) To design an intuitive interface that guides students through defining milestones, tasks, hierarchies, and dependencies without requiring prior project management expertise
(4) To develop an interactive Gantt chart visualisation using D3.js [18, 52], ensuring seamless integration with HWU's existing technological stack
(5) To implement functionality allowing students and supervisors to explore project timelines, track progress, and re-evaluate goals as projects evolve
(6) To provide export capabilities enabling students to include Gantt charts and associated data in their project documentation
(7) To conduct usability evaluation through user studies with students and supervisors to assess the tool's effectiveness

1.3 Organisation

The organisation of this report is as follows:

- Section 2: Presents the background research, discussing related work found in the technical literature and its relevance to the project.
- Section 3: Describes the research methodology, including requirements gathering, system design approach, and evaluation methods.
- Section 4: Examines the technical and practical feasibility of the proposed system, including technology selection and integration with HWU's infrastructure.
- Section 5 Documents the implementation of the Interactive Gantt Chart Builder, detailing the system architecture, database design, core visualisation features, and the key technical decisions made during development.
- Section 7: Summarises the findings, discusses the implications of the work, and outlines potential directions for future research and development.

2 Background Research

2.1 Gantt Charts and Project Management

The Gantt chart traces its origins to the early twentieth century and the scientific management movement. American mechanical engineer and management consultant Henry Gantt first described a version of his charts in 1903, published alongside Frederick W. Taylor's influential work on shop management, though scholars debate whether their widespread adoption began during the First World War [55]. Conceived as production planning tools, Gantt charts monitored worker performance and machine utilisation, embodying the principles of scientific management through rational analysis, clear task durations, and systematic improvement [55]. Crucially, they addressed the factory-wide machine loading problem, without which Taylor's planning office would have been unworkable [55]. While project management applications of Gantt charts were initially limited, their paper-based forms declined in the 1950s and 1960s. They regained popularity with computerised methods in the mid-1960s and were further revitalised by microcomputer project software in the 1980s. Today, Gantt charts have evolved into flexible, software-based tools central to modern project management [55].

Building upon Henry Gantt's original design, modern Gantt charts have evolved to incorporate sophisticated features that support complex project planning. A comprehensive Gantt chart integrates multiple elements: [16, 49]

- A vertical list of tasks or activities on the left side.
- A horizontal timeline indicating project duration (days, weeks, or months)
- Graphical bars representing individual task durations and their temporal placement.
- Lines or arrows indicating dependencies between tasks.
- Labels showing team members or assignees.
- Special markers (typically diamonds) that denote milestones or important project achievements.

Task dependencies are fundamental to effective Gantt chart usage, defining the relationships between tasks and dictating the order in which activities must occur. Project management identifies four primary dependency types [40, 47]. The Finish-to-Start (FS) dependency, where a successor task cannot commence until its predecessor concludes, represents the most common relationship type [37]. Start-to-Start (SS) dependencies allow a task to begin only after another task has also began, enabling parallel workflows that share a start point. Finish-to-Finish (FF) relationships constrain task completion, requiring one task to finish before another can conclude. Finally, the Start-to-Finish (SF) dependency, the least common type, denotes that a task cannot finish until another task begins, typically appearing in handover or transition scenarios.

Beyond dependencies, Gantt charts visualise the critical path the longest sequence of dependent tasks that determines the project's minimum completion time. This enables

project managers to distinguish between tasks that are critical to meeting deadlines and those that possess scheduling flexibility [5]. This distinction proves particularly valuable for resource allocation and risk management decisions.

Gantt charts occupy a distinct position within the broader landscape of project management methodologies. Unlike Kanban boards, which emphasise workflow states and work-in-progress limits while focusing on the volume of work completed, in progress, and not yet started, Gantt charts provide explicit temporal representations of task scheduling and dependencies displayed on a visual timeline [42]. Similarly, while Program Evaluation and Review Technique (PERT) charts focus on identifying critical paths through network diagrams using nodes and arrows to represent task dependencies, Gantt charts offer a more intuitive timeline-based bar chart visualisation that facilitates stakeholder communication and progress tracking [30, 41].

Within project management practice, Gantt charts serve multiple critical functions that extend beyond simple schedule visualisation. As planning tools, they enable project managers to structure complex projects by breaking them into manageable tasks with defined durations and sequences [53]. Their visual representation of timelines, task dependencies, and resource allocation facilitates effective coordination and communication among project stakeholders [53], ensuring all team members maintain a shared understanding of project progress and responsibilities [53]. Furthermore, Gantt charts function as monitoring instruments through features such as baseline comparisons and progress tracking, allowing project managers to identify deviations early and make informed decisions about resource reallocation or timeline adjustments [53]. This combination of planning, communication, and monitoring capabilities has established Gantt charts as one of the most frequently used project management tools, ranking fourth among 70 tools and techniques in a survey of 750 project managers [8, 53].

The application of Gantt charts and project management tools in educational contexts, particularly for individual student projects, remains an underexplored area in academic literature. While Bednjanec and Tretinjak demonstrate educators' use of Gantt charts for planning school activities and curriculum development [7], research on project management tools in higher education focuses predominantly on institutional IT departments [27] and collaborative group work [22]. This focus on institutional and collaborative applications leaves a gap in understanding how individual students might effectively use Gantt charts for dissertation management. Existing project management research centres on professional contexts [8], where tools are designed for team-based environments with different requirements than individual academic projects.

2.2 Gantt Charts in Educational Contexts

Within educational contexts, Gantt charts serve multiple pedagogical functions. For educators, Gantt charts provide a structured approach to planning school activities, curriculum

development, and lesson planning, with their graphical representation of beginning dates, ending dates, and duration parameters proving particularly effective for tracking various educational activities over specific time periods [7]. Beyond their use by educators, Gantt charts offer valuable support for students undertaking complex academic projects such as dissertations. The ability to visualise project timelines and decompose large, complex undertakings into manageable components with clear deadlines proves particularly valuable when managing extended research projects [46]. This visual representation of project structure enables both students and educators to conceptualise and monitor multiple activities across extended timeframes [7].

However, students often encounter challenges when applying Gantt charts to individual academic projects, particularly concerning the maintenance and updating of project timelines throughout extended research periods. Research with PhD students reveals that while Gantt charts can be beneficial for planning, the unpredictable nature of academic research makes it difficult to maintain detailed charts over multi-year projects, with students reporting frustration when plans deviate from initial timelines [2]. This psychological barrier to updating charts is compounded by the investment required: as one PhD student observes, "if one invest a lot of time and energy in creating a detailed Gantt chart, then one is not gonna be willing to adjust this chart when things didn't work out as planned" [2]. This reluctance to revise detailed plans creates a tension between the effort invested in initial planning and the flexibility required for successful academic project management.

Traditional Gantt chart tools required frequent manual updates to remain accurate, which created a significant maintenance burden. Although modern software reduces this through automation, regular updates are still necessary, and increasing project complexity, particularly in large projects like a dissertation, makes Gantt charts challenging to manage [35]. Modern interactive Gantt chart platforms address many of these challenges through features such as drag-and-drop task scheduling, automatic dependency adjustments, and real-time collaborative updates, which significantly reduce the effort required to maintain project schedules and enable more flexible adaptation to changing project requirements [50]. These interactive capabilities prove particularly valuable for complex projects where tasks and timelines must be frequently adjusted, as they allow users to modify schedules efficiently without the cumbersome manual recalculation required by static chart implementations [50].

The distinction between individual and collaborative project contexts significantly influences the application of project management tools like Gantt charts. In team-based academic projects, collaborative project management platforms provide mechanisms for distributed task assignment, shared responsibility, and collective accountability, enabling group members to coordinate schedules, track individual contributions, and maintain visibility of project progress [22]. These collaborative features address common challenges in group work, including coordination costs, workload equity concerns, and the need for clear role definition among team members [22]. However, individual academic projects such as dissertations present fundamentally different challenges. Unlike team projects

where collective objectives and shared evaluation create mutual accountability, dissertation students bear sole responsibility for project completion and are evaluated individually on their work [43]. Research into academic project management reveals that while research involving multiple students may appear collaborative, the structure actually consists of professor-student duos where each student manages their own sub-project as a junior partner rather than as part of an integrated team [43]. This absence of collective objectives and team-based evaluation means individual dissertation students must rely entirely on autonomous use of project management tools without the distributed workload or accountability structures present in genuine team contexts [8]. The challenges become especially pronounced when individual students encounter the psychological barriers associated with maintaining detailed project schedules throughout extended research periods, as the sole responsibility for both planning and plan maintenance creates tension between the effort invested in initial timeline development and the flexibility required for adaptive project management [2].

2.3 Interactive Data Visualisation

Effective data visualisation relies on fundamental principles that ensure clarity and comprehension. Key design principles include knowing the audience, maintaining simplicity, choosing appropriate chart types, using colour strategically whilst maintaining accessibility, and providing proper context through titles, labels and scales [28]. Interactive visualisation differs fundamentally from static visualisation in its approach to user engagement: whilst static visualisation employs an author-driven approach where a single data story is communicated, interactive visualisation adopts a reader-driven approach that enables users to extract multiple views and explore data stories dynamically through inputs such as mouse clicks or keyboard commands [34]. Interactive graphics can provide crucial information that cannot be obtained from static graphics, making them valuable tools for promoting transparency and enabling additional exploration of datasets [54]. The benefits of interactivity in educational contexts are substantial, as:

- Interactive visualisations transform data into engaging learning experiences that enhance comprehension, improve retention, and enable personalised exploration of complex concepts [26].
- User interaction paradigms for visualisation tools encompass several distinct approaches: drag-and-drop interactions enable intuitive grouping, reordering and moving of objects through direct manipulation, providing clear signifiers and immediate visual feedback throughout the interaction [31].
- Form-based interfaces present users with structured options through menus and input fields that guide interaction, allowing menu selection by keying in associated numbers or letters whilst graphical menus employ pointing devices [14].

- Direct manipulation interfaces let users interact with on-screen objects through physical, step-by-step actions that provide instant visual feedback. For example, users can resize a graphic or drag an item to a new position [45].

These interaction paradigms collectively support varied user needs and task requirements, with direct manipulation particularly valued for its ability to exploit perceptual and motor skills whilst reducing reliance on linguistic comprehension [45].

3 Methodology

3.1 Stakeholders and Use Cases

The Interactive Gantt Chart Builder system serves distinct stakeholder groups within the academic environment, each with specific needs and interactions with the system. Understanding these stakeholders and their use cases is essential for establishing clear system boundaries and functional requirements.

The primary stakeholders for Interactive Gantt Chart Builder include:

- **Students:** The primary users who create and manage dissertation project timelines, track progress, and maintain schedules throughout the academic year. The system must accommodate varying levels of experience with project management tools.
- **Supervisors:** Academic staff who monitor student progress through view-only access to Gantt charts. This enables supervisors to assess progress, identify issues, and provide timely guidance across multiple students without administrative burden.
- **Project System Developers:** The technical team responsible for system maintenance and enhancement. The Gantt chart builder must integrate seamlessly with existing HWU infrastructure while maintaining data consistency and extensibility.
- **HWU Project Database:** The external system storing student project information, supervisor assignments, and submission records. Bidirectional data synchronisation ensures consistency across the project management ecosystem.

Use cases are availible in the appendix Section A. Figure **??** in the appendix illustrates the primary use cases for the Interactive Gantt Chart Builder system, organised around the key functional areas identified in the requirements analysis. The use case diagram employs standard UML notation to represent actors, use cases, and their relationships, providing a visual representation of system functionality from the users' perspective.

- **Create Project** represents the foundational use case where students establish their dissertation project with essential metadata including title, description, and dates. The diagram shows this extends to editing project details as understanding evolves.
- **Create Tasks** allows students to break down their dissertation into manageable components with name, description, dates, and duration. The diagram shows this

extends to editing and deleting tasks, with guided input to assist inexperienced users.
- **Create Milestone** enables students to mark significant deliverables and deadlines.
- **Create Task Dependency** enables students to define the task relationships discussed previously [8].
- **Update Task Status** provides progress tracking, allowing students to mark tasks as not started, in progress, or completed.
- **View Project Overview** offers students a comprehensive summary of tasks, milestones, and progress. This viewing capability extends from project creation and connects to Gantt chart visualisation, providing quick status checks for supervision meetings.
- **View Schedule Status** enables students to assess whether they are ahead, on track, or behind schedule based on completion status and dates. The system provides visual indicators highlighting schedule health, helping students develop time management skills through explicit feedback.
- **Add Supervisor Comments** enables supervisors to provide feedback directly within the planning context.
- **Save Data to Project** ensures all project information persists to the HWU database.

The relationships employ standard UML conventions: *extend* indicates optional functionality building upon a base use case, while *include* represents mandatory sub-processes [4].

3.2 Requirement Analysis

The requirement analysis for the Interactive Gantt Chart Builder was conducted through examination of existing literature on project management tools in educational contexts, consultation of stakeholder needs identified through preliminary discussions with the current system developers, and analysis of the challenges students face when managing individual dissertation projects. This process identified five core functional requirements that form the foundation of the proposed system.

The requirements are categorised using the MoSCoW method into Must have (M), Should have (S), Could have (C), and Won't have (W) [39]. To label them they are indicated FR[#] to represent Functional Requirement number [#] and NFR[#] to represent Non-Functional Requirement number [#]. Each requirement has a User Story[13] format description to clarify the requirements purpose and context.

- **FR1 - M - Project Data Creation and Management**
  As a student, I must be able to create and store my project information (title, description, start date, end date) so that I have a foundation for my Gantt chart.
- **FR2 - M - Task Creation and Editing**

As a student, I must be able to create, edit, and delete tasks with names, descriptions, start dates, and end dates so that I can break down my dissertation into manageable components.

- **FR3 - M - Task Dependencies**
  As a student, I must be able to define dependencies between tasks (Finish-to-Start relationships) so that my project plan reflects the logical sequence of work.
- **FR4 - M - Gantt Chart Visualisation**
  As a student, I must be able to view my tasks and milestones as a visual Gantt chart using D3.js so that I can understand my project timeline at a glance.
- **FR5 - M - Data Saving**
  As a student, I must be able to save my project data to a database (SQLite) so that my work is not lost between sessions.
- **FR6 - M - Milestone Creation**
  As a student, I must be able to create milestones with specific dates so that I can mark key deliverables and deadlines in my dissertation timeline.
- **FR7 - S - Guided Task Creation Interface**
  As a student, I should receive contextual help and validation when creating tasks so that I create well-structured, realistic project plans.
- **FR8 - S - Interactive Timeline Adjustment**
  As a student, I should be able to drag and drop tasks on the Gantt chart to adjust dates and durations so that I can quickly adapt my plan as circumstances change.
- **FR9 - S - Automatic Dependency Adjustment**
  As a student, when I modify a task's dates, the system should automatically update dependent tasks so that I maintain a logically consistent schedule.
- **FR10 - S - Progress Tracking and Status Updates**
  As a student, I should be able to mark tasks as not started, in progress, or completed so that I can track my actual progress against my plan.
- **FR11 - S - Progress Indicator and Timeline Comparison**
  As a student, the system should tell me if I am ahead of schedule, on track, or behind schedule so that I can take corrective action when needed.
- **FR12 - S - Gantt Chart Export (PNG/JPEG)**
  As a student, I should be able to export my Gantt chart as an image (PNG/JPEG) so that I can include it in my dissertation appendix and supervision meeting documents.
- **FR13 - S - Supervisor View-Only Access**
  As a supervisor, I should be able to view my students' Gantt charts in read-only mode so that I can monitor their progress and provide guidance.
- **FR14 - S - Task Hierarchy**
  As a student, I should be able to create subtasks under parent tasks so that I can organise complex phases of work hierarchically.
- **FR15 - S - Impossible Dependencies**
  As a student, the system should prevent me from creating logically impossible schedules.
- **FR16 - S - Timeline Zoom and Pan Controls**

As a student, I should be able to zoom in/out and pan across the timeline so that I can view different levels of detail from my entire project to individual weeks.

- **FR17 - S - Hover Tooltips with Task Details**
  As a student, when I hover over a task bar, I should see detailed information (task name, dates, duration, dependencies) so that I can quickly access task details without navigating away.
- **FR18 - C - Kanban Board View**
  As a student, I could have an alternative Kanban board view of my tasks so that I can visualise my work in a different format that shows current work in progress.
- **FR19 - C - Task Categories/Tags**
  As a student, I could assign categories or tags to tasks (e.g., "Research", "Writing", "Data Collection") so that I can filter and organise my work by type.
- **FR20 - C - Multiple Dependency Types**
  As a student, I could define different dependency types (Start-to-Start, Finish-to-Finish, Start-to-Finish) beyond the basic Finish-to-Start so that I can model more complex task relationships.
- **FR21 - C - Critical Path Highlighting [8]**
  As a student, I could view the critical path highlighted on my Gantt chart so that I can identify which tasks have no scheduling flexibility [8].
- **FR22 - C - Supervisor Comment and Feedback System**
  As a supervisor, I could add comments and feedback on specific tasks or the overall project plan so that I can provide guidance directly within the tool.
- **NFR1 - M - Web Browser Compatibility**
  As a student, I must be able to access and use the Gantt chart builder on any modern web browser (Chrome, Firefox, Safari, Edge) without installing additional plugins or software so that I can work on my project plan from any computer.
- **NFR2 - M - Data Security and GDPR Compliance**
  As a student, the system must store all my project data securely and be GDPR-compliant.
- **NFR3 - S - Response Time Performance**
  As a student, the system should render my Gantt chart and respond to my interactions within 2 seconds (for projects with up to 100 tasks) so that I can work efficiently without frustrating delays that discourage regular use.
- **NFR4 - S - Intuitive User Interface**
  As a student, the system should provide a clear, intuitive interface that I can use effectively without extensive training so that I can focus on planning my dissertation rather than learning how to use the tool.
- **NFR5 - S - Mobile-Friendly Responsive Design**
  As a student, the system should adapt its interface for my tablet and mobile devices so that I can view and ideally edit my project plan while away from my computer.
- **NFR6 - C - Accessibility Compliance**

As a student with disabilities, the system could meet WCAG 2.1 Level AA accessibility standards with keyboard navigation, screen reader support, and sufficient color contrast so that I can use the tool effectively regardless of my accessibility needs.

- **NFR7 - C - Scalability for Multiple Concurrent Users**
  As a student, the system could support at least 200 concurrent users without performance degradation so that I can access my Gantt chart reliably even during peak usage periods like the start of semester or project deadlines

3.3 Evaluation Method

System testing will be employed to validate that the implemented system satisfies its specified functional requirements (FR1-FR22) and non-functional requirements (NFR1-NFR7). System testing is appropriate for this project as it evaluates the complete, integrated system against its requirements specification [19], which is essential for verifying that the web-based application functions correctly as a whole rather than merely as isolated components.

The system testing approach will utilise a requirements traceability matrix that maps each functional requirement to specific test cases, ensuring comprehensive coverage. This approach builds on the concept of requirements traceability as discussed by Gotel and Finkelstein [21]. Test cases will be designed to verify both positive scenarios (expected use cases) and negative scenarios (error handling and validation), following the principles outlined by Myers [36]. Particular attention will be given to the interactive features that distinguish this system from static project management tools.

Critical areas for system testing include:

- **Data persistence and integrity**: Verifying that project data, tasks, dependencies, and milestones are correctly saved to and retrieved from the SQLite database (FR5).
- **Dependency management**: Validating that task dependencies function correctly, including automatic adjustment of dependent tasks when parent tasks are modified (FR3, FR9) and prevention of logically impossible schedules (FR15).
- **Interactive visualisation**: Confirming that the D3.js-based Gantt chart accurately represents task data and responds correctly to user interactions such as drag-and-drop timeline adjustments (FR4, FR8).
- **Progress tracking functionality**: Testing the status update mechanisms and schedule comparison features to ensure accurate tracking of project progress (FR10, FR11).

Performance testing will be conducted to validate NFR3, measuring system response times under various conditions including different numbers of tasks (up to 100) and multiple concurrent users. Browser compatibility testing across Chrome, Firefox, Safari, and Edge will verify NFR1, while responsive design testing on various screen sizes will assess NFR5.

Accessibility evaluation will be conducted against WCAG 2.1 Level AA standards, ensuring compliance with requirements such as keyboard navigation, screen reader support, and sufficient color contrast. In line with these standards, Garrido et al. [17] demonstrate how client-side refactoring can enhance usability for visually impaired users, underscoring the importance of adaptable interfaces in meeting accessibility needs. Together, these evaluations ensure that the system is tested not only for performance and compatibility but also for adaptability and accessibility, providing a comprehensive validation of the non-functional requirements.

Usability evaluation is essential for assessing whether the system achieves its core objective of helping students create comprehensive project plans through an intuitive, easy-to-use interface (NFR4) [51]. The usability study will recruit participants from the target user population-honours students at Heriot-Watt University who are planning or currently undertaking dissertation projects.

The usability evaluation will employ a task-based assessment methodology where participants complete a series of realistic scenarios using the Interactive Gantt Chart Builder [25]. These scenarios will be designed to exercise key functional requirements including:

- Creating a new project and defining initial project parameters (FR1)
- Adding tasks with dependencies and milestones (FR2, FR3, FR6)
- Using interactive features to adjust the timeline (FR8)
- Tracking progress and interpreting schedule status (FR10, FR11)
- Exporting the Gantt chart for documentation purposes (FR12)

Quantitative usability metrics will be collected during these tasks, including:

- Task completion rates and time-on-task measurements
- Number of errors encountered during task completion
- System Usability Scale (SUS) scores to provide standardised usability assessment [11]

Qualitative feedback will be gathered through post-task questionnaires and semi-structured interviews, focusing on participants' perceptions of the system's usefulness for dissertation planning, ease of learning (addressing NFR4), and the value of interactive features compared to static planning approaches. The questionnaire will be designed to gather specific feedback on:

- The clarity and helpfulness of the guided task creation interface (FR7)
- The effectiveness of the visual Gantt chart representation (FR4)
- The usefulness of interactive timeline adjustment compared to form-based editing (FR8)
- Overall satisfaction with the tool for managing dissertation projects

This combination of requirements-based system testing and user-centered usability evaluation provides comprehensive assessment of both technical correctness and practical utility, ensuring that the system not only functions as specified but also effectively supports students in planning their honours projects [12].

4 Feasibility

4.1 Stakeholder Meetings

The technical infrastructure and integration requirements were established through consultation with Kayvan Karim, a key stakeholder in the university's honours project management system. Karim provided guidance on technical constraints and integration approach while leaving the specific design and implementation details open-ended, with no prescribed visual requirements or timeline expectations.

While the existing honours project system architecture uses CHTML, there are no restrictions on using standard HTML and modern web technologies for the Gantt chart component. Integration will occur through a dedicated access point on the existing projects page, positioned above the ethics form section. The system will inherit existing authentication infrastructure using local email-based login.

4.2 Tools

The system will be implemented using ASP.NET MVC framework with Entity Framework for database operations. SQLite will serve as the database solution. The frontend will use standard HTML and JavaScript, with D3.js providing interactive visualisation capabilities for drag-and-drop interactions and dynamic chart rendering.

4.3 Interface Mock-ups

Preliminary interface mock-ups were created using D3.js to visualise the layout and interaction flow of the Gantt chart builder, drawing inspiration from examples in Data Visualisation Resources and The Chartmaker Directory [3, 33].

The mock-up matches the current HWU honours project system's visual style, ensuring consistency. It illustrates the user workflow from project setup through task definition to timeline interaction, demonstrating how visual elements such as milestones (amber diamonds) and tasks (coloured horizontal bars) may appear in the final implementation.

4.4 Limitations and Risks

The primary limitations of this project relate to scope and evaluation constraints. The system will focus exclusively on individual honours project management, with collaborative features and advanced dependency types (Start-to-Start, Finish-to-Finish) deferred to future development. The evaluation will be conducted with a limited sample of honours students within a single academic cycle, which may not capture long-term usability patterns or the full range of dissertation project types.

Technical risks are minimal given the use of established technologies (ASP.NET MVC, Entity Framework, D3.js) and straightforward integration with existing HWU infrastructure. The main implementation risk involves ensuring the interactive visualisation performs adequately with complex project schedules containing numerous tasks and dependencies, though preliminary testing with D3.js mock-ups suggests this is manageable within the specified performance requirements (NFR3).

5 Implementation and Contributions

The system was implemented as a fully independent, locally-hosted web application using Node.js with Express, SQLite, and D3.js, diverging from the ASP.NET MVC stack proposed during the feasibility phase. This decision was taken following further technical consultation and reflection on the project's actual scope. The original ASP.NET proposal was made under the assumption that the Gantt chart builder would need to integrate directly with the existing HWU honours project system at projects.hw.ac.uk. As the system was ultimately developed as a standalone prototype, that integration constraint no longer applied, and a reassessment of the technology stack was warranted.

Node.js with Express proved to be a more natural fit for several reasons. The most significant is that the entire application, from the server-side API through to the D3.js frontend visualisation, is written in JavaScript. This means there is no language boundary between the backend and the frontend, which simplifies development considerably and keeps the codebase cohesive. Node.js is also particularly well suited to the kind of iterative, prototype-driven work this project involved, offering rapid development cycles through its lightweight architecture and the breadth of the npm package ecosystem [32]. In practical terms, this gave direct access to `better-sqlite3`, a synchronous SQLite binding that integrates cleanly with Express without the need for a separate ORM layer such as Entity Framework. On balance, Node.js and Express were the more pragmatic choice for a time-constrained individual project of this scale.

5.1 System Architecture

The application follows a client-server architecture, with an Express server handling all REST API endpoints and a SQLite database managed via the `sqlite3` library providing persistent storage. The frontend consists of static HTML, CSS, and JavaScript files served directly by Express. Authentication uses a local email-based login system without passwords, with sessions managed via `localStorage` on the client side. The system is divided into two main modules: a login and dashboard module (`src/login-system`) and the Gantt chart builder module (`src/gantt-builder`), each with their own public assets and server-side routes. This separation keeps concerns distinct and allows each module to be reasoned about independently.

Authentication is intentionally lightweight, reflecting the system's role as a prototype that inherits its user base from the existing HWU honours project system. Rather than implementing a separate registration and password system, users log in by entering only their email address. The server looks this up against the `students` table and, if a matching record is found, returns the student's details along with a `role` field set to `'student'`. If no match is found in the students table, the same lookup is performed against the `supervisors` table, returning `role: 'supervisor'` on a successful match. The returned user object is stored in `localStorage` on the client side and used throughout the session to gate access to edit functionality and to scope data fetches to the correct student. This approach was agreed upon during stakeholder consultation with Dr Kayvan Karim, who confirmed that a simple email-based login was appropriate given that student records would already exist in the project system prior to use.

5.2 Database Design and Data Persistence

The database schema, defined in `src/login-system/database/setup.js`, implements the core entities identified during requirements analysis. The schema comprises eight tables: `students`, `student_projects`, `tasks`, `dependencies`, `subtasks`, `supervisors`, `supervisor_projects`, and `comments`. This directly addresses the data persistence requirement **FR5** and underpins all other data-related requirements.

The `students` table records each user's email address and display name, serving as the authentication source for the email-based login system. The `student_projects` table stores the title, description, start date, and end date for each project, satisfying **FR1**. Projects are associated with students via a foreign key on `student_id`, supporting a many-to-many relationship through this junction table.

The `tasks` table stores the task name, description, start and end dates, a `progress_percentage` field enforced by a `CHECK` constraint to the range 0–100, an `is_milestone` integer flag, a `colour` field for tag-derived styling, a `display_order` field, and a `tag` field for category assignment. A `parent_task_id` column provides a self-referencing foreign

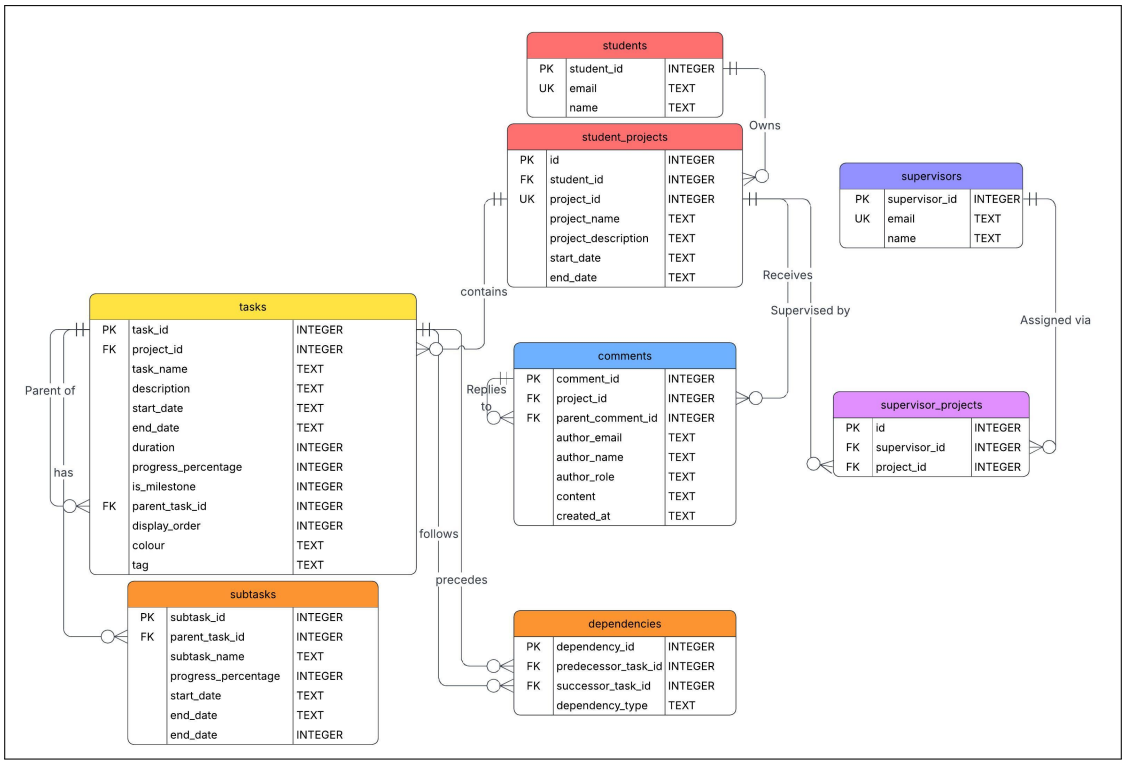


Fig. 1. Entity-Relationship Diagram.

key back to `tasks(task_id)`, supporting hierarchical grouping. This single table design supports **FR2**, **FR6**, **FR10**, **FR14**, and **FR19**. The `dependencies` table stores predecessor and successor task IDs alongside a `dependency_type` field (defaulting to `'finish-to-start'`), supporting **FR3**. A separate `subtasks` table holds fine-grained subtask data (name, dates, progress, and a `parent_task_id` referencing the parent task) keeping subtask records distinct from top-level tasks. An excerpt of the core table definitions is shown below:

```
1
2  CREATE TABLE IF NOT EXISTS tasks (
3      task_id             INTEGER PRIMARY KEY AUTOINCREMENT,
4      project_id          INTEGER NOT NULL,
5      task_name           TEXT    NOT NULL,
6      start_date          TEXT,
7      end_date            TEXT,
8      progress_percentage INTEGER DEFAULT 0
9          CHECK(progress_percentage >= 0 AND progress_percentage <=
       100),
10     is_milestone        INTEGER DEFAULT 0,
11     parent_task_id      INTEGER,
12     display_order       INTEGER DEFAULT 0,
13     colour              TEXT    DEFAULT '#4a90d9',
```

```
    tag                 TEXT    DEFAULT NULL,
    FOREIGN KEY (project_id)     REFERENCES student_projects(project_id),
    FOREIGN KEY (parent_task_id) REFERENCES tasks(task_id)
);

CREATE TABLE IF NOT EXISTS dependencies (
    dependency_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    predecessor_task_id INTEGER NOT NULL,
    successor_task_id   INTEGER NOT NULL,
    dependency_type     TEXT    NOT NULL DEFAULT 'finish-to-start',
    FOREIGN KEY (predecessor_task_id) REFERENCES tasks(task_id),
    FOREIGN KEY (successor_task_id)   REFERENCES tasks(task_id)
);
```

Code Listing 1. Core table definitions from setup.js

A noteworthy design consideration is that SQLite has no native date type; all dates are stored as `TEXT` in `YYYY-MM-DD` ISO 8601 format [48]. This means date parsing is handled on the client side using D3's `d3.timeParse('%Y-%m-%d')` when data is loaded, and dates are serialised back to strings with `d3.timeFormat('%Y-%m-%d')` before being sent in API payloads.

The `comments` table stores comment content, author email, author name, role (constrained to `'student'` or `'supervisor'` via a `CHECK`), a creation timestamp defaulting to `datetime('now')`, and an optional `parent_comment_id` self-reference for threaded replies, supporting **FR22**.

Rather than running a separate migration tool, the server applies idempotent schema changes on startup using `ALTER TABLE ... ADD COLUMN` wrapped in error handlers that silently ignore `duplicate column name` errors. This allowed new columns such as `start_date` and `end_date` on `student_projects` to be added to an existing database without requiring a full reset.

All database interactions occur through REST API endpoints exposed by the Express server. The client communicates with these endpoints via the browser's `fetch` API, issuing `GET` requests to retrieve data and `POST`, `PATCH`, and `DELETE` requests to create, update, and remove records respectively. For example, loading a project's tasks and dependencies on page initialisation is performed in parallel using `Promise.all`:

```
    const [dbTasks, dbDependencies, dbSubtasks, projectBounds] = await Promise.all([
    fetchTasks(projectId),
    fetchDependencies(projectId),
    fetchSubtasks(projectId),
```

```
    fetchProjectBounds(projectId)
]);
```

Code Listing 2. Parallel data fetch on page load (interactive-chart.js)

5.3 Gantt Chart Visualisation

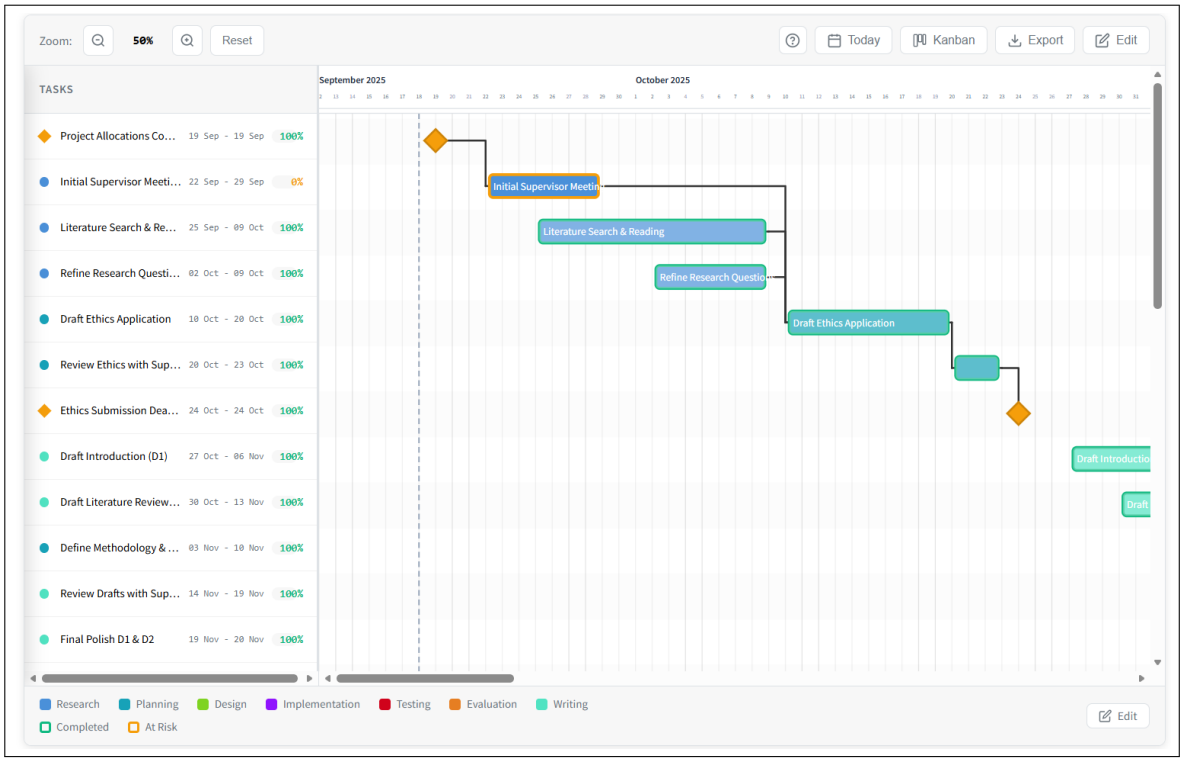


Fig. 2. Screenshot of the Interactive Gantt Chart Builder's dashbord

The core visualisation is implemented as a `GanttChart` class in `src/gantt-builder/public/js/gantt-chart.js`, addressing **FR4**. The chart is constructed as a multi-layer SVG element using D3.js [18, 52]. Layers are appended in z-order to a main <g> chart area to ensure correct visual stacking: row backgrounds, grid lines, dependency lines, a drop preview layer, task bars, task labels, the today indicator line, project boundary markers, dependency handle circles, and a dependency drag layer. The axis groups responsible for the day and month headers are appended directly to the root SVG element rather than to the scrolling chart area, which is what produces the sticky header behaviour as the user scrolls vertically. Most layers use a keyed D3 data join pattern (`.data(tasks, d => d.id).join(enter, update, exit)`) [9, 18, 52] so that only changed elements are updated on re-render, rather than rebuilding the entire SVG on every state change. The dependency lines layer is an exception: because each dependency

is composed of three sub-elements (two connector segments and a stepped path), it clears its contents before each redraw rather than relying on a pure enter/update/exit cycle.

Horizontal positioning is handled by a `d3.scaleTime()`[9, 18, 52] scale that maps dates to pixel positions, while vertical positioning uses a `d3.scaleBand()`[9, 18, 52] scale that maps task IDs to row heights with configurable padding. A daily tick axis is generated via `d3.axisTop()`[9, 18, 52], and month boundary labels are produced by iterating over dates returned by `d3.timeMonths()`[9, 18, 52]. Both scales are constructed together whenever the data or zoom level changes, as shown below:

```
// X Scale: Time scale mapping dates to pixel positions
this.scaleX = d3.scaleTime()
    .domain(this.dateExtent)
    .range([0, this.chartWidth]);

// Y Scale: Band scale mapping task IDs to row positions
this.scaleY = d3.scaleBand()
    .domain(this.data.map(d => d.id))
    .range([0, this.chartHeight])
    .padding(0.2);
```

Code Listing 3. Scale initialisation in GanttChart (gantt-chart.js)

Smooth transitions are applied to bar position and width changes using `.transition().duration(500)` [9, 18, 52], providing visual feedback when data is updated. The progress overlay on each bar is also animated, with its width computed as a proportion of the full bar width:

```
bars.select('.task-bar')
    .transition().duration(500)
    .attr('x', d => this.scaleX(d.startDate) + this.pixelShaving)
    .attr('y', d => this.scaleY(d.id) +
        (this.scaleY.bandwidth() - this.barHeight) / 2)
    .attr('width', d => Math.max(0,
        this.scaleX(d.endDate) - this.scaleX(d.startDate)
        - (this.pixelShaving * 2)));

bars.select('.task-bar-progress')
    .transition().duration(500)
    .attr('width', d => {
        const fullWidth = this.scaleX(d.endDate) - this.scaleX(d.
   startDate)
            - (this.pixelShaving * 2);
        return Math.max(0, fullWidth * ((d.progress || 0) / 100));
    });
```

Code Listing 4. Animated bar and progress overlay update (gantt-chart.js)

Dependency lines are drawn using a D3 path generator configured with the `d3.curveStepAfter` interpolator, which produces the characteristic right-angled routing between predecessor and successor bars:

```
const lineGenerator = d3.line()
    .curve(d3.curveStepAfter)
    .x(d => d.x)
    .y(d => d.y);
```

Code Listing 5. Dependency line path generator (gantt-chart.js)

Milestones (**FR6**) are rendered as diamond-shaped `<polygon>` elements rather than rectangles, centred on their single date position. A darker stroke derived using `d3.color(hex).darker(0.5)` [9, 18, 52] is applied to the diamond outline to improve visual contrast against the fill. For regular task bars, the stroke colour is determined separately by a status function: a green border (`#10b981`) indicates a completed task (100% progress), and an amber border (`#f59e0b`) indicates a task that is overdue but not yet complete, directly supporting the schedule status indicator requirement **FR11**. Task categories (**FR19**) are reflected through bar fill colours, with seven fixed tags; Research, Planning, Design, Implementation, Testing, Evaluation, and Writing. Each of these tags are mapped to a distinct hex colour via a `TAG_COLOURS` constant defined in `interactive-chart.js`.

5.4 Interactive Timeline Adjustment

Drag-and-drop timeline adjustment (**FR8**) is implemented using four separate `d3.drag()`[9, 18, 52] behaviours: one for moving an entire task bar, one for left-edge resizing (adjusting the start date), one for right-edge resizing (adjusting the end date), and one for milestone dragging. All drag behaviours are gated behind an `editMode` flag so that dragging is only active when the user has explicitly enabled editing. Date snapping is achieved by converting pixel deltas to day deltas using `Math.round(totalDelta / dayWidth)` and then advancing dates via `d3.timeDay.offset(originalDate, daysDelta)`[9, 18, 52], ensuring tasks always snap to whole-day boundaries. An `rAF`-based auto-scroll loop activates when the cursor approaches the horizontal edges of the scroll container, allowing users to drag tasks beyond the visible viewport.

New tasks can also be placed on the chart directly via a drag-to-drop interaction from the slide-out add-task panel. When the user drags the task preview element from the panel, a floating `div` ghost element is appended to the document body and tracks the cursor via `mousemove` (Figure 3). When the cursor crosses into the chart scroll container, the ghost acquires an `over-chart` CSS class and `ganttChart.showDropPreview()`[9, 18, 52] is simultaneously called to render a second, SVG-native preview inside the `dropPreview` layer. This SVG preview displays a semi-transparent dashed bar (or diamond for milestones) snapped to the nearest row, with a subtle full-width row highlight behind it. The target row is resolved

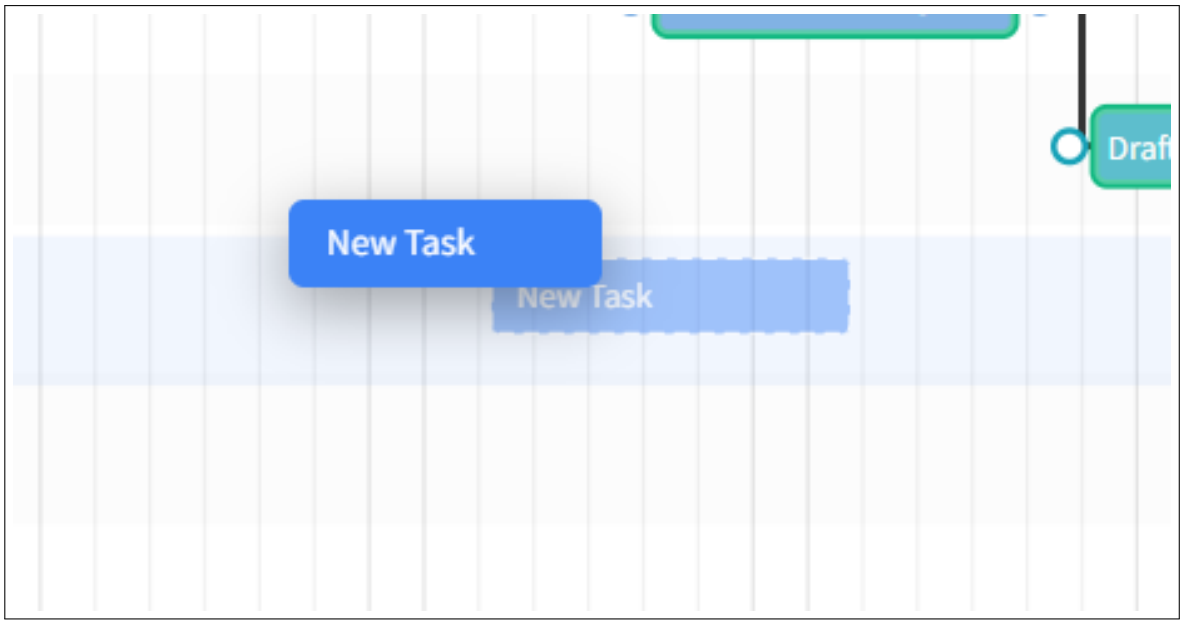


Fig. 3. Screenshot of the drag and drop ghost task.

by dividing `cursorY` by `rowHeight` and flooring the result, and the start date is computed in real time by inverting the cursor's X position through `ganttChart.scaleX.invert()`[9, 18, 52], rounded to the nearest day via `d3.timeDay.round()`[9, 18, 52]. On release, the drop position is used to initialise the new task's start date, with the end date defaulting to seven days later.

Timeline zoom (**FR16**) is controlled through a `dayWidth` property that defines the number of pixels rendered per day. Zoom level is expressed as a percentage relative to a default `dayWidth` of 35px. Zooming in increments this percentage by 25 points, zooming out decrements it by 25 points, and a reset function restores 100%. After each zoom change the chart re-renders, recomputing `d3.scaleTime()`'s [9, 18, 52] output range from the updated `dayWidth` and recalculating the total SVG width as `dayCount * dayWidth` [9, 18, 52], causing all bars, labels, and grid lines to reflow accordingly. A "Go to Today" button computes the scroll offset via `scaleX(today)` and centres the current date in the viewport.

5.5 Task Dependencies

Finish-to-Start task dependencies (**FR3**) are created interactively in edit mode by dragging from circular handle elements that appear at the left and right edges of each task bar. The interaction is implemented using a custom three-phase event approach rather than `d3.drag()`: a `mousedown` [9, 18, 52] listener on the handle circles begins the drag and appends a rubber-band line to a dedicated SVG layer; native `document` listeners for `mousemove` and `mouseup` then track cursor movement and finalise the connection respectively. Mouse

coordinates are converted to SVG space using `svg.node().getBoundingClientRect()`, and `document.elementFromPoint()` [1] is used on `mouseup` to identify the target handle the user released onto.

The connection type is resolved from which sides were joined: dragging from a left handle to another left handle produces a start-to-start dependency, while any other combination produces the more common finish-to-start type. Before a dependency is committed, two validation checks are applied. First, a depth-first search cycle detection algorithm (`wouldCreateCycle`) traverses the current dependency adjacency list to ensure the new edge would not form a circular relationship. Second, a creation-time violation check rejects any dependency that would be immediately violated. For example, a finish-to-start dependency where the predecessor already ends after the successor starts. These checks together partially satisfy **FR15**. If both pass, the dependency is added to the in-memory graph and rendered immediately. Dependencies involving tasks that have not yet been saved to the database (identified by temporary negative IDs) are staged in a `pendingNewDependencies` array and only POSTed to the server after those tasks receive real IDs on save.

Dependency lines are redrawn on every drag frame during task movement (**FR9**) by calling `#updateDependencyLines(0)` with a zero transition duration to provide real-time visual feedback without animation overhead. Violated dependencies apear when a predecessor ends after its successor starts, they are detected per-line and styled as orange dashed lines via the `dependency-violated` CSS class. In edit mode, hovering over a dependency line highlights it red and clicking it removes it, both from the in-memory graph and, on save, from the database. On saving, all remaining violations are checked and the user must confirm before changes are committed, further supporting **FR15**.

5.6 Progress Tracking and Schedule Status

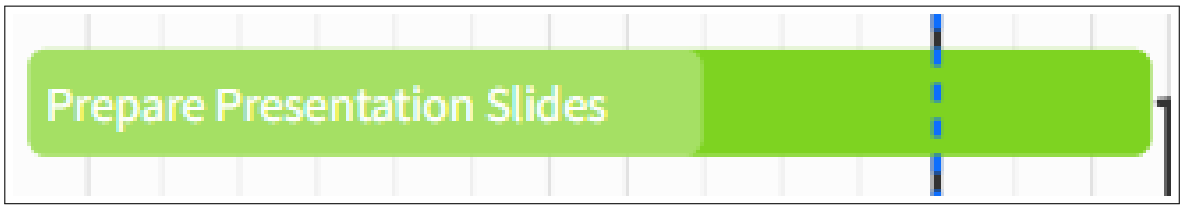


Fig. 4. Screenshot of the Interactive Gantt Chart Builder's Progress Tracking on a Task

Progress tracking (**FR10**) is implemented through a numeric `progress_percentage` field on each task, editable via a number input in the task list panel. The progress value drives a semi-transparent white overlay on the task bar whose width is proportional to the completion percentage Figure 4, animated using `.transition().duration(500)`. The same colour logic is mirrored in the task list panel, where the progress input's text colour changes to reflect status providing a consistent at-a-glance status across both views. The

colours are denoted as the following; green for complete, amber for overdue, and grey (`var(--text-secondary)`) otherwise.

Schedule status (**FR11**) is determined by the `#getTaskStatusStyle(task)` method: a task is considered completed if `progress === 100`, shown with a green border (`#10b981`), and at risk if `progress < 100` and the end date has passed today, shown with an amber border (`#f59e0b`). Tasks with neither condition return no border style. A today indicator line, rendered via a single-element D3 data join (`.data([today]).join('line')`) [9, 18, 52], provides a persistent vertical reference for the current date, accompanied by a `'Today'` label above the axis. The indicator is only rendered when the current date falls within the chart's visible date extent; if the user scrolls or zooms to a range that does not include today, the element is removed entirely.

5.7 Task Hierarchy and Subtasks

Subtask support (**FR14**) is implemented through a `subtasks` database table and a client-side `subtaskData` Map that associates subtask arrays with their parent task IDs. Expand state is tracked via an `expandedTasks` Set, with all tasks collapsed by default on load. A `buildFlatList()` function interleaves subtasks beneath their expanded parents to produce the single ordered array consumed by both `GanttChart` and `TaskList` on every render. To avoid ID collisions on the shared `d3.scaleBand()` domain, subtask objects are assigned synthetic IDs of `1 000 000 + subtask_id`, ensuring they occupy distinct row positions without conflicting with top-level task IDs. Subtasks inherit their parent task's colour, providing a clear visual grouping on the chart.

Subtask bars are rendered at a reduced height of 14px compared to 24px for parent task bars. An expand/collapse chevron on parent rows controls visibility. In edit mode, an "Add Subtask" marker row appears after the last subtask of an expanded parent, providing a clear interaction affordance. Subtask additions, deletions, and name or date changes are all staged in pending collections during edit mode and only committed to the database on Save, consistent with the rest of the edit mode transaction model.

5.8 Additional Features

Hover tooltips (**FR17**) are managed by a `TooltipManager` class that wraps a pre-existing `<div>` element and positions it relative to the cursor using `event.clientX` and `event.clientY` offsets. The tooltip displays the task name, formatted start and end dates via `d3.timeFormat('%d %b %Y')`[9, 18, 52], and current progress. For milestones, only the single milestone date is shown and the progress field is omitted.

Gantt chart export (**FR12**) (Figure 9) is implemented by constructing a combined SVG element in memory that embeds inline CSS, clones the live chart SVG, and reconstructs the task list as SVG text elements. This combined SVG is serialised with `XMLSerializer`,

drawn onto a <canvas> at twice the resolution for sharpness, and exported as either a **PDF** via jsPDF or a **JPEG** via `canvas.toBlob()`.

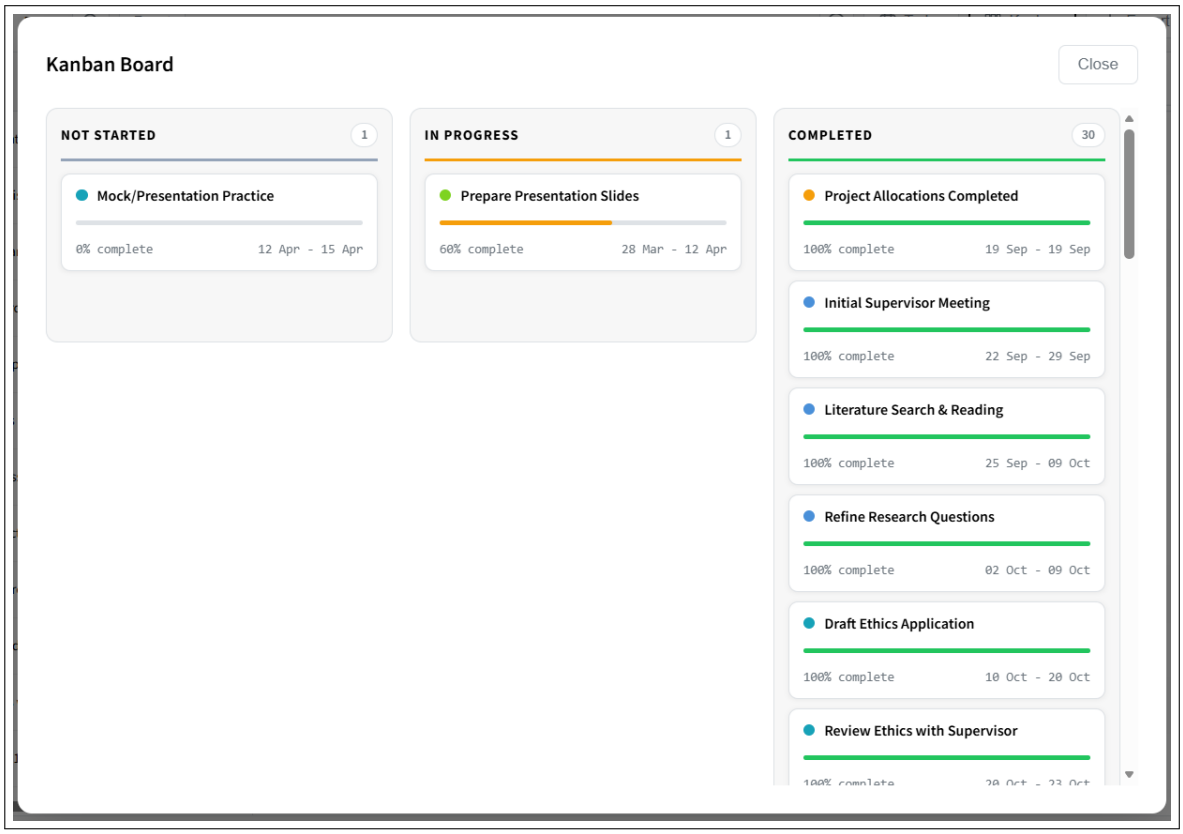


Fig. 5. Screenshot of the Interactive Gantt Chart Builder's Kanban Board View

A Kanban board view (**FR18**) was implemented as a modal overlay that organises tasks into three columns based on progress: Not Started (0%), In Progress (1–99%), and Completed (100%) (Figure 5). Each card includes a range slider whose progress fill and percentage label update live while dragging; on release, the new value is PATCHed to the database, the Gantt bar is updated in place, and the board re-renders so the card moves to the correct column immediately.

Supervisor view-only access (**FR13**) is enforced by the login endpoint, which checks the `students` table first and falls back to the `supervisors` table, returning a `role` field in the response. When `role === 'supervisor'`, the edit mode button is hidden and no drag or edit interactions are wired, ensuring supervisors can only read the chart.

A guided task creation interface (**FR7**) is provided through a slide-out panel in edit mode with inline validation that prevents end dates being set before start dates. A contextual help button opens an eight-slide tutorial modal, each slide rendering a live mini-Gantt

chart using real `GanttChart` instances populated with hardcoded demo data, illustrating system concepts in context.

5.9 Requirements Coverage Summary

All six Must Have functional requirements (FR1–FR6) were fully implemented. Nine of the eleven Should Have functional requirements were fully implemented. FR7 (guided task creation interface) was partially addressed: inline date validation is present in the add-task panel, and a contextual help button opens an eight-slide tutorial modal that teaches users how to operate the system. However, a fully guided step-by-step task creation wizard (one that actively directs users through each field in sequence) was not implemented; the tutorial serves as a reference guide rather than an in-context creation assistant. FR9 (automatic dependency adjustment) was also partially addressed: dependency violations are highlighted in real time during drag using orange dashed lines, giving users immediate visual feedback, but fully cascading date shifts to downstream successor tasks were not implemented.

Of the five Could Have requirements, FR18, FR19, and FR22 were fully implemented. FR20 (additional dependency types) was partially addressed. The dependency handle architecture naturally produces two types: dragging from a right handle to any handle creates a finish-to-start dependency, while dragging from a left handle to another left handle creates a start-to-start dependency. The rendering and violation detection logic is structured as a branching dispatch on `dependencyType`, making it straightforward to extend with further types:

```
if (dep.dependencyType === 'start-to-start') {
    // route line from predecessor start to successor start
} else {
    // default: finish-to-start
    // route line from predecessor end to successor start
}
```

Code Listing 6. Dependency type branching in `#updateDependencyLines` (gantt-chart.js)

Despite this extensibility, finish-to-finish and start-to-finish types were deliberately deferred. Finish-to-start is the dependency type that most naturally aligns with the chart's linear time axis and ascending start-date sort order, expressing the unambiguous constraint that one task must complete before another begins. Supporting additional types would require users to reason about less intuitive constraints adding cognitive load without a clear benefit for the target user group of individual dissertation students. Keeping the dependency model focused on the most common type produced a more streamlined and accessible interaction. FR21 (critical path highlighting) was deferred as noted in the original MoSCoW analysis.

Both Must Have non-functional requirements (NFR1–NFR2) were addressed through browser-standard technologies and local data storage. The Should Have non-functional requirements NFR3, NFR4, and NFR5 were targeted throughout development, with NFR5 representing a partial implementation given the chart's horizontal scrolling nature on small screens.

5.10 Bug Fixes and Edge Cases

The majority of the bugs described in this section were identified during the usability evaluation study, where participants interacting with the system under realistic conditions exposed edge cases that had not surfaced during development testing. All identified issues were resolved following the conclusion of the user study and prior to the final stakeholder demonstration with Dr Kayvan Karim. Several bugs were identified and resolved during development. The first concerned the save pipeline for subtasks added to tasks that had not yet been persisted to the database. New tasks are assigned a temporary negative client-side ID until saved; subtasks captured this temporary value and froze it as their `parent_task_id`. When the save ran, the server assigned real IDs to the new tasks but the pending subtask entries still referenced the stale negative value, causing a 404 on `/api/tasks/-1/subtasks` and aborting the save. The fix records the old temporary ID before overwriting it, then patches any pending subtask entries that still reference it. A related issue meant that a failed save followed by a retry would re-POST all items in the pending arrays, including those already written on the first attempt, producing duplicate rows. This was resolved by converting both save loops to `while` loops that shift each item off the array immediately after a confirmed successful POST, so retries only process items not yet persisted.

A further issue allowed subtasks to be added to milestone tasks, which represent single-point-in-time events and have no meaningful duration for child tasks. The fix conditionally omits the add-subtask button when `d.isMilestone` is true, removes the corresponding click handler from milestone rows, and adds a defensive early return in the `setOnAddSubtask` callback.

A timing bug was found in the tutorial auto-open flow triggered by a `?new=true` URL parameter on project creation. Because `interactive-chart.js` is `async` and suspends at its first `await`, `tutorial.js` completed its synchronous `init()` during that suspension window without rendering slide content. When `interactive-chart.js` later made the modal visible, the first slide was blank. The fix moves responsibility for auto-opening entirely into `tutorial.js`, which reads the URL parameter directly, calls `openTutorial()`, and invokes `goToSlide(0)` in the correct order.

After saving a newly created task, two client-side data structures, `expandedTasks` and `subtaskData`, remained keyed on the old temporary ID, causing subtasks to become orphaned and the task to appear collapsed after save. The post-save migration was extended to remap both structures to the real database ID. Additionally, attempting to delete an

unsaved task issued a `DELETE` request to the server unnecessarily, returning a 404. The fix checks whether `task.id` is negative and, if so, removes the task purely from client-side state without making any network request.

A minor visual bug was also found in the Kanban board progress slider, where the drag handle appeared above the track rather than centred on it. In WebKit-based browsers the `::-webkit-slider-thumb` pseudo-element is not automatically centred on a custom-height track. The fix applies an explicit `margin-top` of `-5px`, derived from the formula $-\frac{thumbHeight - trackHeight}{2} = -\frac{14-4}{2}$, to align the 14px thumb with the 4px progress track.

6 Evaluation

This section presents the evaluation of the Interactive Gantt Chart Builder across three complementary strands: requirements-based system testing against the functional and non-functional requirements defined in Section 3.2, a user study with honours students employing the System Usability Scale (SUS) [11] alongside task-based and qualitative feedback, and stakeholder demonstrations with the project supervisor (Dr Pierre Le Bras) and Dr Kayvan Karim.

6.1 Requirements Testing

System testing was conducted informally against the MoSCoW requirements defined in Section 3.2, verifying each requirement through direct interaction with the running application. All six Must Have functional requirements (FR1–FR6) were confirmed as fully implemented and functioning correctly. Project creation and management (FR1), task creation, editing and deletion (FR2), Finish-to-Start dependencies (FR3), D3.js-based Gantt chart visualisation (FR4), SQLite data persistence (FR5), and milestone creation (FR6) all behaved as specified under both normal and edge-case conditions.

Of the eleven Should Have requirements, nine were verified as fully complete. FR7 (guided task creation interface) was marked as partially complete: inline date validation and a contextual eight-slide tutorial modal are present, but a fully guided step-by-step creation wizard was not implemented. FR9 (automatic dependency adjustment) was also partially complete, as dependency violations are highlighted visually in real time during drag, but fully cascading date shifts to downstream tasks were not implemented. All remaining Should Have requirements were fully verified, covering drag-and-drop adjustment (FR8), progress tracking (FR10), schedule status indicator (FR11), export (FR12), supervisor view-only access (FR13), task hierarchy (FR14), prevention of impossible schedules (FR15), zoom and pan (FR16), and hover tooltips (FR17).

Of the five Could Have requirements, FR18 (Kanban board), FR19 (task categories and tags), and FR22 (supervisor comment and feedback system) were fully implemented and

verified. FR20 (additional dependency types) was partially addressed through the start-to-start interaction described in Section 5.5, and FR21 (critical path highlighting) was deferred as planned.

Both Must Have non-functional requirements were satisfied. NFR1 (browser compatibility) was verified across Chrome, Firefox, and Edge, and NFR2 (data security and GDPR compliance) was met through local data storage and the use of anonymised fictitious data during evaluation. NFR3 (response time) was satisfied in practice, with the chart rendering and responding to interactions well within the two-second threshold for all tested project sizes. NFR4 (intuitive interface) and NFR5 (mobile-friendly design) were assessed through the usability study described below, with NFR5 representing a partial implementation due to the chart's horizontal scrolling behaviour on smaller screens.

6.2 User Study

6.2.1 Participants and Study Design. Fifteen participants were recruited from the target population of honours students at Heriot-Watt University, all of whom had prior experience creating project plans for university assignments or dissertations. Approximately ten participants reported regular or occasional prior use of Gantt chart tools such as TeamGantt, Microsoft Project, or Excel, while the remaining five had little or no prior experience and relied primarily on informal to-do lists. The majority rated themselves as either Very Confident or Extremely Confident in using computers, with one participant reporting no confidence at all, which provided a useful spread of technical backgrounds across the group.

The study employed an exploratory think-aloud methodology [25]. Participants were given an empty project and invited to interact with the system freely, without instruction, while verbalising their thoughts and actions. The researcher observed and took notes throughout, recording any areas of difficulty, confusion, or unexpected behaviour. This free-exploration approach was supplemented by a small number of directed prompts: if a participant had not naturally discovered a feature by a certain point in the session, they were asked to attempt specific activities, including creating a milestone, creating a task, creating a dependency between tasks, navigating and reading a pre-populated Gantt chart, using the hover tooltips, finding and enabling edit mode, renaming and repositioning tasks, using the Kanban board progress view, and exporting the chart. The time taken to complete each activity and the correctness of the outcome were noted by the researcher. Following the hands-on session, the database was also checked to confirm whether the data created by participants had been persisted correctly. Participants then completed the ten-item SUS questionnaire [11], a set of background experience and demographic questions, self-reported task difficulty ratings for each feature, and a set of open-ended feedback questions.

The SUS was selected as the primary quantitative instrument for this study for several reasons. It is one of the most widely used and psychometrically well-validated post-study

usability questionnaires available, offering strong reliability, validity, and sensitivity to differences between systems, and it requires no licence fee [44]. It is also notably sensitive at relatively small sample sizes, making it well-suited to a study of fifteen participants [44]. Scoring followed the standard procedure described by Sauro and Lewis [44]: for each odd-numbered item the score contribution is the scale position minus one, and for each even-numbered item it is five minus the scale position, giving each item a contribution between zero and four. The sum of all ten contributions is then multiplied by 2.5 to produce a final score on a scale of zero to one hundred [44].

6.2.2 SUS Results. The individual SUS scores for the fifteen participants were: 77.5, 80.0, 100.0, 92.5, 85.0, 100.0, 97.5, 90.0, 67.5, 100.0, 85.0, 97.5, 70.0, 75.0, and 100.0. The mean SUS score was **87.8**, which corresponds to an adjective rating of **Excellent** on Bangor et al.'s scale [6]. Scores ranged from 67.5 to 100.0, with four participants awarding the maximum possible score and eleven of fifteen scoring 80.0 or above.

This result suggests that the Interactive Gantt Chart Builder substantially exceeds typical usability expectations for a first-iteration prototype. The design decisions made throughout development, particularly the edit mode interaction model, the contextual tutorial, and the visual clarity of the chart, all appear to have contributed positively to the overall usability of the system (NFR4). For context, an average SUS score across a wide range of systems sits at approximately 68, with scores above 80 considered above average and scores above 85 considered excellent [6][44]. The system's mean of 87.8 therefore places it well above both of these thresholds, which is an encouraging result for a prototype tool aimed at students with varying levels of prior project management experience.

6.2.3 Task Difficulty Ratings. Self-reported task difficulty ratings were broadly positive across all features, with the vast majority of tasks rated as either Very Easy or Easy. The features participants found most straightforward were opening the Kanban board, creating tasks, dragging and resizing tasks on the timeline, and working through the tutorial. These results are encouraging as they cover several of the core interactive features of the system, suggesting that the drag-and-drop interaction model (FR8) and the Kanban board view (FR18) were well designed for discoverability and ease of use.

The features participants found relatively more challenging were milestones, task dependencies, progress bar usage, and adding and managing subtasks. Several participants noted an initial unfamiliarity with the drag interaction that resolved quickly once they began using it, suggesting a minor learning curve rather than a fundamental usability problem. One participant recorded a Neutral rating for a specific feature and one indicated they had not used a particular task during the session, both of which fall within the normal variation expected across a group of fifteen participants with differing backgrounds.

6.2.4 Qualitative Feedback. The tutorial modal received consistently positive feedback from participants. Comments described it as "brilliant" and noted that it "gave me confidence"

and "helped a lot." This is a particularly meaningful result given that the tutorial was implemented as part of FR7 specifically to address the challenge of onboarding users who have no prior project management experience. The positive reception supports the view that even a reference-style guide, rather than a fully guided wizard, can be effective in reducing the initial barrier to engaging with an unfamiliar tool [46].

Participants also offered a number of constructive suggestions for future development. The most commonly raised request was the ability to customise task colours independently of the predefined tag categories, reflecting a desire for greater visual personalisation. Other suggestions included grouping the edit controls together in the interface, moving the edit mode toggle higher up the page for faster access, and improving the subtask user interface to make the task hierarchy clearer. These suggestions sit well alongside the task difficulty data, where subtask management was identified as one of the more challenging aspects of the system, and they provide a clear set of actionable improvements for future iterations of the tool.

6.3 Stakeholder Evaluation

6.3.1 Supervisor Feedback. Following the user study, feedback was received from the project supervisor identifying two issues for resolution prior to the final stakeholder demonstration. The first was a bug whereby subtasks did not move with their parent task when the parent was dragged to a new position on the timeline. The second was a visual inconsistency in the Kanban board where the progress slider drag handle did not always appear, which affected the usability of the progress update interaction. Both issues were resolved before the final meeting with Dr Kayvan Karim, as described in Section 5.10.

6.3.2 Stakeholder Demonstration with Dr Kayvan Karim. A final demonstration of the completed system was conducted with Dr Kayvan Karim, the key technical stakeholder for the HWU honours project system at projects.hw.ac.uk. Dr Karim's overall assessment was positive. He confirmed that the system is well-suited for integration and that the database structure aligns closely with the schema used by the existing project system, which would simplify the integration process considerably.

Dr Karim outlined two practical pathways through which the Gantt chart builder could be brought into the existing system. The first is to host it as a separate, independently-deployed application that is linked to from the current project pages. This approach requires fewer changes to the existing codebase but would require the database layer to be adapted to handle the scale of the live system's datasets. The second option is to integrate the builder directly within the existing project site, which would take advantage of the current architecture but would require more substantial work to bridge the JavaScript-based frontend with the C# backend used by the existing system. Dr Karim noted that this language difference was the primary technical challenge in either scenario, though it was considered manageable rather than a blocker. Integration work is planned to begin in mid-April 2026.

Dr Karim also suggested a number of features worth considering for future development, including a search bar and filter mechanism based on task type or tag, and milestone grouping functionality. These suggestions align naturally with the Could Have requirements deferred during this phase of development and provide a useful starting point for the next iteration of the system.

6.4 Evaluation Summary

Taken together, the evaluation provides strong evidence that the Interactive Gantt Chart Builder meets its stated objectives. All Must Have requirements were fully implemented and verified, nine of eleven Should Have requirements were fully delivered with two partially addressed, and three of five Could Have requirements were completed. The mean SUS score of 87.8 places the system in the Excellent category on Bangor et al.'s adjective scale [6], well above the commonly cited average of 68, and the qualitative feedback highlighted the tutorial and the drag-and-drop interaction model as particular strengths. Subtask management and dependency creation were identified as areas for improvement in future iterations. The positive stakeholder assessment from Dr Karim, combined with a confirmed integration timeline beginning in April 2026, validates both the technical feasibility of the system and its potential to provide genuine value within the HWU honours project ecosystem.

7 Conclusion

This dissertation has designed, developed, and evaluated an Interactive Gantt Chart Builder specifically for honours project students at Heriot-Watt University. The background research demonstrates that whilst Gantt charts rank as the fourth most widely used project management tool among professionals [8], their application in individual student dissertation contexts remains underexplored in academic literature.

The proposed system addresses genuine challenges faced by students managing substantial individual projects over extended periods [46]. Through systematic requirement analysis using MoSCoW prioritisation [39], this research identified 22 functional and 7 non-functional requirements organised as user stories [13]. These encompass core capabilities including project data management, task dependencies, interactive D3.js-based visualisation [18], progress tracking, and supervisor access.

The feasibility analysis confirmed technical viability, and the system was subsequently implemented using a Node.js and Express backend with a SQLite database and a D3.js SVG-based frontend. This stack was chosen for its lightweight footprint, ease of local deployment, and compatibility with the data structures used by HWU's existing project management infrastructure. The implementation delivers all six Must Have functional requirements and nine of the eleven Should Have requirements in full, with the remaining

two partially addressed. Three of the five Could Have requirements were also completed, including a Kanban board view, task categories and tags, and a supervisor comment system.

The system was evaluated through a fifteen-participant think-aloud user study with honours students, requirements-based system testing, and a stakeholder demonstration with Dr Kayvan Karim, the technical lead for the HWU project management system. The mean SUS score of 87.8 places the system in the Excellent category on Bangor et al.'s adjective scale [6], well above the commonly cited average of 68 [44]. Qualitative feedback was particularly positive regarding the tutorial modal and the drag-and-drop interaction model, whilst subtask management and dependency creation were identified as areas for future improvement. Dr Karim's assessment confirmed that the database structure aligns closely with the existing HWU system and that integration is technically feasible, with work planned to begin in mid-April 2026.

Several limitations of the current work should be acknowledged. The system was evaluated as a standalone prototype; usability within the fully integrated HWU environment may differ once the JavaScript frontend must interact with the existing C# backend, and this cannot be assessed until integration is complete. The email-only authentication model is a deliberate simplification appropriate for a prototype that inherits its user base from the existing project system, but it is not production-ready and would require a more robust mechanism before live deployment. Automatic cascading of dates to downstream tasks when a predecessor is moved was not implemented, meaning users must manually resolve dependency violations rather than having the chart adapt for them. Mobile usability also remains a partial limitation, as the horizontal scrolling nature of the chart is suited to desktop use but offers a reduced experience on smaller screens. The user study, whilst sufficient in scale for SUS evaluation, was conducted with fifteen participants from a single department at Heriot-Watt University within a single academic cycle, which may not fully represent the range of dissertation students across disciplines or institutions, and the presence of an observer during the think-aloud sessions may have introduced some degree of performance bias.

Despite these limitations, the evaluation results offer meaningful insight into the design decisions made throughout development. A mean SUS score of 87.8 for a first-iteration prototype with no prior user testing is an encouraging result, and suggests that the core interaction model was well-calibrated for the target audience. The strong reception of the tutorial modal is especially significant: it validates the decision to implement a reference-style guide rather than a fully guided wizard, demonstrating that lightweight onboarding can meaningfully reduce the barrier to entry for users with no prior project management experience. The free-exploration study design also revealed a clear discoverability gradient across the feature set: task creation, drag-and-drop timeline editing, and the Kanban board were discovered and used with little difficulty, whilst subtask management and dependency creation consistently required more guidance. This pattern provides direct, actionable direction for the next iteration of the interface. Finally, Dr Karim's confirmation that the database schema already aligns with HWU's existing project system is a significant result

in itself, as it means the integration pathway does not require a schema redesign and substantially reduces the technical risk of the next phase of development.

Future work will focus on integrating the builder into the live HWU project management system, addressing the participant feedback through improved subtask and dependency interfaces, and extending the feature set with task colour customisation, a search and filter mechanism, and milestone grouping functionality as suggested during the stakeholder demonstration.

References

[1] W3Schools 2026. *HTML DOM Element getBoundingClientRect() Method.* W3Schools. https://www.w3schools.com/jsref/met_element_getboundingclientrect.asp Accessed: 2026-04-09.

[2] Academiac. 2018. *Are Gantt charts useful for PhD students?* https://academiac.net/2018/10/23/are-gantt-charts-useful-for-phd-students/ Published on academiac.net, Accessed: 14/11/2025.

[3] Andrew Witherley Andy Kirk. [n. d.]. The Chartmaker Directory. https://chartmaker.visualisingdata.com/. Accessed: 25/09/2025.

[4] ArchiMetric. 2024. *Comprehensive Guide on Use Cases and the Concepts of Extend and Include.* https://www.archimetric.com/comprehensive-guide-on-use-cases-and-the-concepts-of-extend-and-include/ Accessed: 19/11/2025.

[5] Asana. 2025. What is a Gantt Chart? Guide to Project Timelines. https://asana.com/resources/gantt-chart-basics. Accessed: 12/11/2025.

[6] Aaron Bangor, Philip Kortum, and James Miller. 2009. Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale. *Journal of Usability Studies* 4, 3 (2009), 114–123.

[7] Andrea Bednjanec and Martina Filipovic Tretinjak. 2013. Application of Gantt charts in the educational process. In *2013 36th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO).* MIPRO, 631–635.

[8] Claude Besner and Brian Hobbs. 2008. Project management practice, generic or contextual: A reality check. *Project Management Journal* 39, 1 (2008), 16–33. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/pmj.20033 doi:10.1002/pmj.20033

[9] Mike Bostock. 2026. *D3.js: Data-Driven Documents.* https://d3js.org/ Accessed: 2026-04-09.

[10] British Computer Society. 2025. *Code of Conduct for BCS Members.* BCS, The Chartered Institute for IT. https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ Accessed: 19/11/2025.

[11] John Brooke. 1995. SUS: A quick and dirty usability scale. *Usability Eval. Ind.* 189 (11 1995).

[12] Tomás Castillo, Benjamín Chamorro, Ignacio Catalán, Pablo Schwarzenberg, Luis Rojas, and Fei Luo. 2025. A Serious Game Approach for Teaching Requirements Engineering: User Experience Evaluation. In *Social Computing and Social Media*, Adela Coman and Simona Vasilache (Eds.). Springer Nature Switzerland, Cham, 20–36.

[13] Mike Cohn. 2009. *User Stories Applied: For Agile Software Development.* Addison-Wesley, Boston.

[14] Methodist Collage. [n. d.]. UNIT-I: Interaction Paradigms: Computing Environments, Analysing Interaction Paradigms, Interaction Frameworks and Styles. https://methodist.edu.in. Human-Computer Interaction Course Materials, Methodist College. Accessed: 17/11/2025.

[15] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). https://eur-lex.europa.eu/eli/reg/2016/679/oj Official Journal of the European Union.

[16] Academy for International Modern Studies (AIMS). [n. d.]. What is Gantt Chart Project Management? https://aims.education/what-is-gantt-chart-project-management/. Accessed: 08/11/2025.

[17] Alejandra Garrido, Sergio Firmenich, Gustavo Rossi, Julián Grigera, Nuria Medina-Medina, and Ivana Harari. 2013. Personalized Web Accessibility using Client-Side Refactoring. *IEEE Internet Computing* 17 (07 2013), 58–66. doi:10.1109/MIC.2012.143

[18] GeeksforGeeks. 2025. *D3.js in JavaScript.* https://www.geeksforgeeks.org/javascript/d3-js/ Accessed: 25/09/2025.

[19] GeeksforGeeks. 2025. *Difference between Unit Testing and System Testing.* https://www.geeksforgeeks.org/software-testing/difference-between-unit-testing-and-system-testing/ Accessed: 19/11/2025.

[20] GeeksforGeeks. 2025. What is Gantt Chart in Project Management? https://www.geeksforgeeks.org/business-studies/what-is-gantt-chart-in-project-management/. Last updated: 23 Jul 2025. Accessed: 08/11/2025.

[21] O.C.Z. Gotel and C.W. Finkelstein. 1994. An analysis of the requirements traceability problem. In *Proceedings of IEEE International Conference on Requirements Engineering*. 94–101. doi:10.1109/ICRE.1994.292398

[22] Hela Hassen. 2024. The use of a project management tool in distance education to enhance students' engagement in group work. *Journal of Learning Development in Higher Education* 30 (Mar. 2024). doi:10.47408/jldhe.vi30.1146

[23] HM Government. 2010. Equality Act 2010. https://www.legislation.gov.uk/ukpga/2010/15/contents UK Public General Acts.

[24] HM Government. 2018. Data Protection Act 2018. https://www.legislation.gov.uk/ukpga/2018/12/contents UK Public General Acts.

[25] Shah Rukh Humayoun, Yael Dubinsky, Tiziana Catarci, Eli Nazarov, and Assaf Israel. 2012. Using a High Level Formal Language for Task Model-Based Usability Evaluation. In *Information Systems: Crossroads for Organization, Management, Accounting and Engineering*, Marco De Marco, Dov Te'eni, Valentina Albano, and Stefano Za (Eds.). Physica-Verlag HD, Heidelberg, 199–207.

[26] Innovare. 2023. Interactive Data Visualization in Education: A Complete Guide. https://innovaresip.com/resources/blog/interactive-data-visualization-in-education-guide/. Accessed: 17/11/2025.

[27] Charles Johnston and David Wierschem. 2007. Project Management Practices in the Information Technology Departments of Various Size Institutions of Higher Education. *Journal of International Technology and Information Management* 16 (01 2007). doi:10.58729/1941-6679.1209

[28] Justinmind. 2025. Principles and examples to master data visualization. https://www.justinmind.com/blog/data-visualization-examples-principles/. Accessed: 17/11/2025.

[29] Andrew Kirkpatrick, Joshue O Connor, Alastair Campbell, and Michael Cooper. 2025. Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

[30] Peter Landau. 2024. Gantt Chart vs. PERT Chart vs. Network Diagram: What Are the Differences? https://www.projectmanager.com/blog/gantt-vs-pert-vs-network-diagram. Published on projectmanager.com, Accessed: 14/11/2025.

[31] Page Laubheimer. 2020. Drag-and-Drop: How to Design for Ease of Use. https://www.nngroup.com/articles/drag-drop/. Nielsen Norman Group. Accessed: 18/11/2025.

[32] Leobit. 2025. ASP.NET Core vs Node.js: Finding the Best Fit for Your App Development. https://leobit.com/blog/asp-dot-net-core-vs-node-js/. Accessed: 26/03/2026.

[33] Visualising Data Ltd. 2025. Data Visualisation Resources. https://visualisingdata.com/resources/. Accessed: 25/09/2025.

[34] Kirti Mahajan and Leena Gokhale. 2018. Comparative Study of Static and Interactive VisualizationApproaches. *International Journal on Computer Science and Engineering* 10 (03 2018), 85–91. doi:10.21817/ijcse/2018/v10i3/181003016

[35] Eddie Meardon. 2025. *Gantt chart: Key features and benefits.* https://www.atlassian.com/agile/project-management/gantt-chart Published on atlassian.com, Accessed: 14/11/2025.

[36] Glenford J. Myers, Corey Sandler, and Tom Badgett. 2012. *The Art of Software Testing* (3rd ed.). John Wiley & Sons. doi:10.1002/9781119202486

[37] Adrian Neumeyer. [n. d.]. Finish-to-Start Dependency Explained (with Examples). https://www.tacticalprojectmanager.com/finish-to-start-dependency/. Accessed: 12/11/2025.

[38] OWASP Foundation. 2021. *OWASP Top 10:2021.* https://owasp.org/Top10/ The Open Web Application Security Project, Accessed: 19/11/2025.

[39] ProductPlan. 2020. What is MoSCoW Prioritization Method? Definition, Overview, and Best Practices. YouTube video. https://www.youtube.com/watch?v=pm4GbSRMElc&t=1s Accessed: 18/11/2025.

[40] ProjectManager. 2021. Gantt Chart Dependencies: Understanding Task Dependency Types. https://www.projectmanager.com/blog/gantt-chart-dependencies. Accessed: 12/11/2025.

[41] Diana Ramos. 2021. PERT Charts vs. Gantt Charts. https://www.smartsheet.com/content/pert-vs-gantt. Published on smarsheet.com, Accessed: 14/11/2025.

[42] Lulu Richter. 2025. Gantt Chart vs. Kanban Board. https://www.smartsheet.com/content/gantt-vs-kanban. Published on smarsheet.com, Accessed: 14/11/2025.
[43] Hélène Riol and Denis Thuillier. 2015. Project management for academic research projects: Balancing structure and flexibility. *International Journal of Project Organisation and Management* 7 (01 2015), 251. doi:10.1504/IJPOM.2015.070792
[44] Jeff Sauro and James R. Lewis. 2012. Chapter 8 - Standardized Usability Questionnaires. In *Quantifying the User Experience*, Jeff Sauro and James R. Lewis (Eds.). Morgan Kaufmann, Boston, 185–240. doi:10.1016/B978-0-12-384968-7.00008-4
[45] Samyukta Sherugar and Raluca Budiu. 2024. Direct Manipulation: Definition. https://www.nngroup.com/articles/direct-manipulation/. Nielsen Norman Group. Accessed: 17/11/2025.
[46] Katie Shives. 2015. *Using Project Management Approaches to Tame Your Dissertation.* https://www.insidehighered.com/blogs/gradhacker/using-project-management-approaches-tame-your-dissertation Accessed: 08/11/2025.
[47] Smartsheet. 2020. *Gantt Chart with Dependencies.* https://www.youtube.com/watch?v=U4QftGiN5Nk Uploaded on YouTube by Smartsheet. Accessed: 30/09/2025.
[48] SQLite Development Team. [n. d.]. Datatypes In SQLite. https://www.sqlite.org/datatype3.html. Accessed: 17/03/2026.
[49] Coursera Staff. 2025. Gantt Charts: What They Are and How to Make Them. https://www.coursera.org/articles/gantt-charts. Accessed: 08/11/2025.
[50] TeamHub. 2023. *Discover the Benefits of an Interactive Gantt Chart Platform.* https://teamhub.com/blog/discover-the-benefits-of-an-interactive-gantt-chart-platform/ Published on TeamHub, Accessed: 14/11/2025.
[51] Prokopia Vlachogianni and Nikolaos Tselios. 2023. Perceived Usability Evaluation of Educational Technology Using the Post-Study System Usability Questionnaire (PSSUQ): A Systematic Review. *Sustainability* 15 (08 2023), 12954. doi:10.3390/su151712954
[52] W3Schools. 2025. *D3.js Tutorial.* https://www.w3schools.com/js/js_graphics_d3js.asp Accessed: 25/09/2025.
[53] Kanika Wadhwa. 2024. *The role of Gantt chart in project management.* Master's thesis. Vaasa University of Applied Sciences.
[54] Tracey L. Weissgerber, Vesna D. Garovic, Marko Savic, Stacey J. Winham, and Natasa M. Milic. 2016. From Static to Interactive: Transforming Data Visualization to Improve Transparency. *PLOS Biology* 14, 6 (06 2016), 1–8. doi:10.1371/journal.pbio.1002484
[55] James M. Wilson. 2003. Gantt charts: A centenary appreciation. *European Journal of Operational Research* 149, 2 (2003), 430–437. doi:10.1016/S0377-2217(02)00769-5 Sequencing and Scheduling.

A Use Case Diagrams

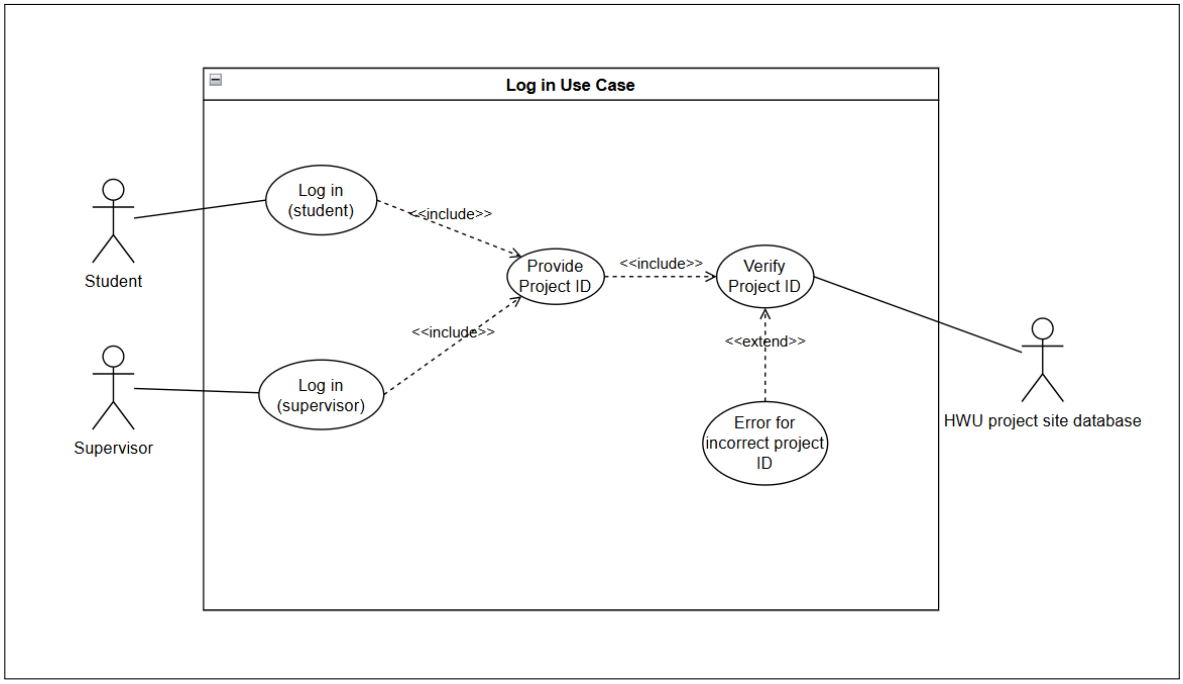


Fig. 6. Log in Use Case Diagram.

B Mock Up

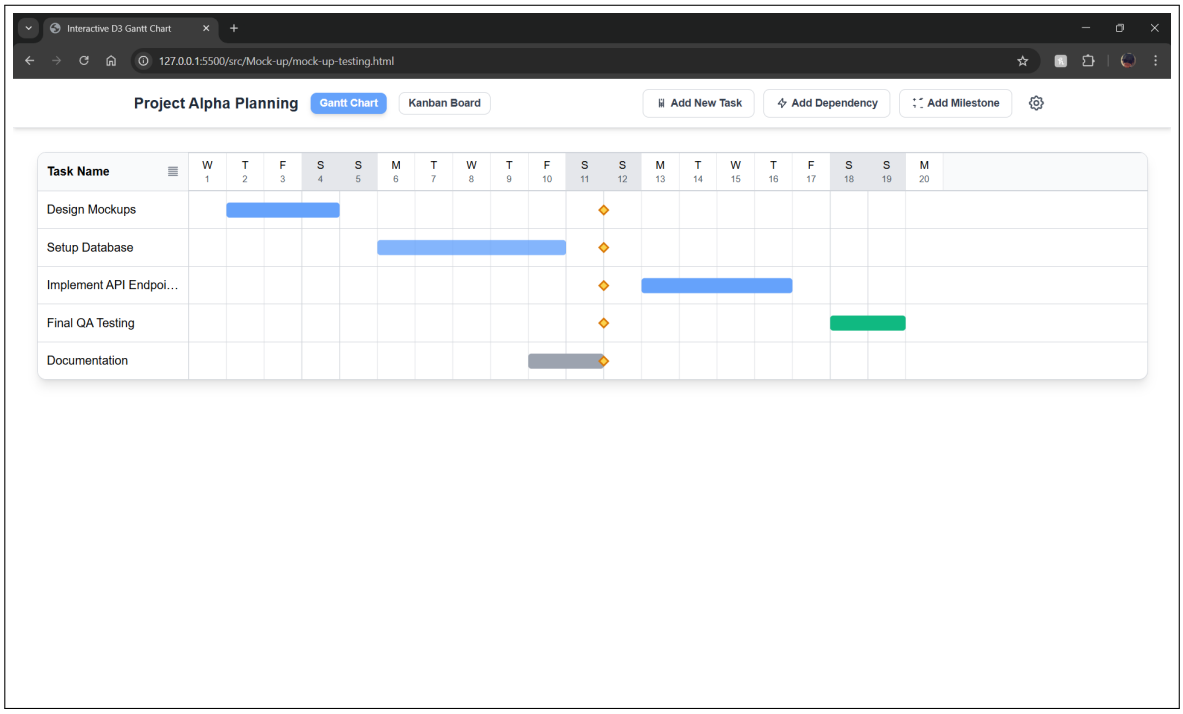


Fig. 7. Rough Mock-ups of the Interactive Gantt Chart Builder Interface.

C Project Management

C.1 Gantt Chart

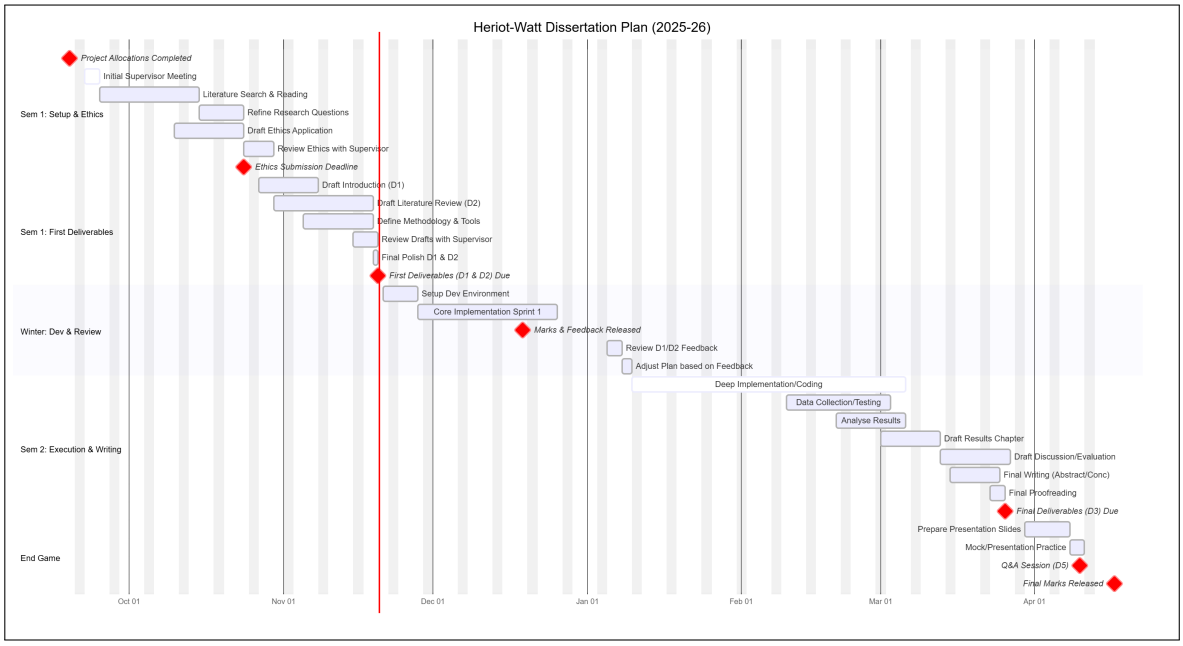


Fig. 8. Gantt Chart for Project Management.

Legend: Red diamonds are milestones; Light blue boxes are tasks; White boxes are supper tasks.

Legend: Orange diamonds are milestones; Coloured boxes are tasks with custom colours representing each task type; Black lines represent dependency lines between tasks.

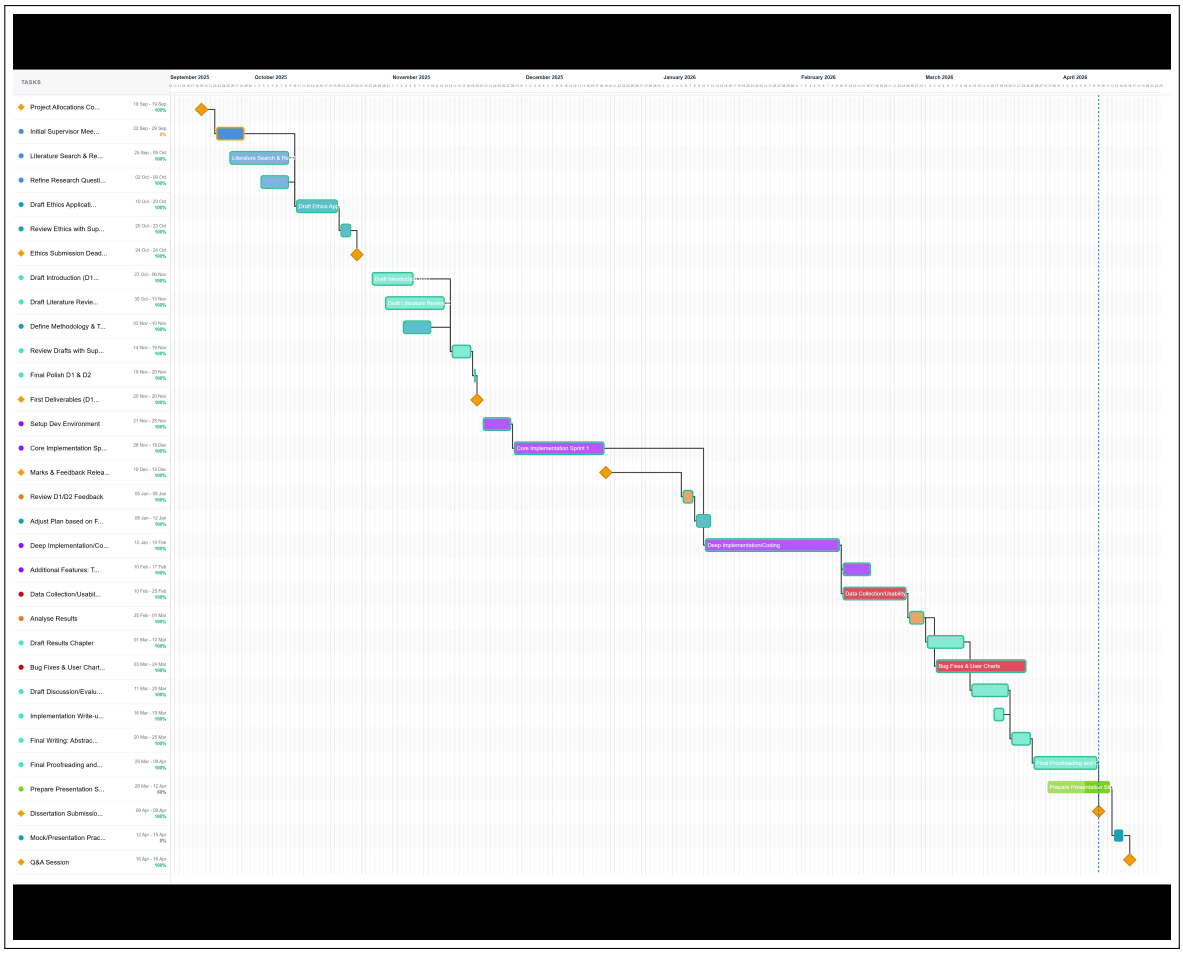


Fig. 9. Gantt Chart

C.2 Risks

The development of the Interactive Gantt Chart Builder involves several identified risks, each assessed for likelihood and impact, with appropriate mitigation strategies.

Task / Milestone	**Date / Period**
Semester 1: Setup & Ethics	
Project Allocations	19 Sep 2025
Initial Supervisor Meeting	22 Sep 2025
Literature Search	25 Sep - 09 Oct
Draft Ethics Application	10 Oct - 20 Oct
Ethics Submission (Deadline)	**24 Oct 2025**
Semester 1: First Deliverables	
Draft Introduction (D1)	27 Oct - 06 Nov
Draft Lit Review (D2)	30 Oct - 13 Nov
Review Drafts	15 Nov - 19 Nov
First Deliverables (D1/D2) (Deadline)	**20 Nov 2025**
Dev Environment Setup	21 Nov 2025
Marks & Feedback	19 Dec 2025
Semester 2: Implementation & Writing	
Review Feedback	05 Jan 2026
Deep Implementation	10 Jan – 20 Feb
Data Collection	10 Feb – 25 Feb
Draft Results Chapter	01 Mar – 10 Mar
Draft Discussion	11 Mar – 20 Mar
Final Polish (D3)	21 Mar – 25 Mar
Final Dissertation (D3) (Deadline)	**26 Mar 2026**
End Game	
Viva Preparation	30 Mar – 09 Apr
Q&A Session (D5)	**10 Apr 2026**
Final Marks Released	17 Apr 2026
Expo (Optional) (D6)	TBC

Table 1. Project Schedule

Task / Milestone	Date / Period	Tag
Semester 1: Setup & Ethics		
Project Allocations Completed	**19 Sep 2025**	Planning
Initial Supervisor Meeting	22 Sep 2025	Planning
Literature Search & Reading	25 Sep - 9 Oct 2025	Research
Refine Research Questions	2 Oct - 9 Oct 2025	Research
Draft Ethics Application	10 Oct - 20 Oct 2025	Planning
Review Ethics with Supervisor	20 Oct - 23 Oct 2025	Planning
Ethics Submission Deadline	**24 Oct 2025**	Planning
Semester 1: First Deliverables		
Draft Introduction (D1)	27 Oct - 6 Nov 2025	Writing
Draft Literature Review (D2)	30 Oct - 13 Nov 2025	Writing
Define Methodology & Tools	3 Nov - 10 Nov 2025	Planning
Review Drafts with Supervisor	14 Nov - 19 Nov 2025	Writing
Final Polish D1 & D2	17 Nov - 20 Nov 2025	Writing
First Deliverables (D1 & D2) Due	**20 Nov 2025**	Writing
Setup Dev Environment	21 Nov - 28 Nov 2025	Implementation
Core Implementation Sprint 1	28 Nov - 19 Dec 2025	Implementation
Winter: Dev & Review		
Marks & Feedback Released	**19 Dec 2025**	Evaluation
Review D1/D2 Feedback	5 Jan - 9 Jan 2026	Evaluation
Adjust Plan based on Feedback	8 Jan - 12 Jan 2026	Planning
Deep Implementation/Coding	10 Jan - 20 Feb 2026	Implementation
Login System Implementation	19 Jan - 26 Jan 2026	Implementation
Gantt Chart Page & Dependencies	26 Jan - 3 Feb 2026	Implementation
Progress Bar & Core Features	3 Feb - 10 Feb 2026	Implementation
Export Button & Styling	10 Feb - 17 Feb 2026	Implementation
Additional Features: Tags, Subtasks, Slidable Tasks	17 Feb - 24 Feb 2026	Implementation
Semester 2: Execution & Writing		
Data Collection/Usability Testing	10 Feb - 25 Feb 2026	Testing
Analyse Results	20 Feb - 1 Mar 2026	Evaluation
Tutorial, Supervisor Mode & Comment System	2 Mar - 3 Mar 2026	Implementation
Bug Fixes & User Charts	3 Mar - 24 Mar 2026	Testing
Draft Results Chapter	1 Mar - 10 Mar 2026	Writing
Implementation Write-up	16 Mar - 19 Mar 2026	Writing
Draft Discussion/Evaluation	11 Mar - 20 Mar 2026	Writing
Final Writing: Abstract & Conclusion	20 Mar - 25 Mar 2026	Writing
Final Proofreading	24 Mar - 26 Mar 2026	Writing
Final Deliverables (D3) Due	**26 Mar 2026**	Writing
First Draft Submitted to Supervisor	**29 Mar 2026**	Writing

Table 2. Project Schedule - Revised (Used withe the gantt chart made with the builder)

Task / Milestone	Date / Period	Tag
End Game		
Prepare Presentation Slides	28 Mar - 7 Apr 2026	Design
Mock/Presentation Practice	3 Apr - 8 Apr 2026	Planning
Q&A Session (D5)	**9 Apr 2026**	Evaluation
Final Marks Released	**17 Apr 2026**	Evaluation

Table 3. Project Schedule - Revised (Used withe the gantt chart made with the builder)

Risk	Level	Mitigation Strategy
Technical Implementation	Medium	Preliminary D3.js mock-ups validate feasibility. Iterative development with regular testing will identify performance issues early. Established technologies reduce implementation uncertainty.
Integration with HWU System	Low-Medium	Early stakeholder consultation established clear technical constraints. System inherits existing authentication infrastructure. Regular communication with developers will address emerging concerns.
Timeline Delays	Medium	Project schedule incorporates buffer time. MoSCoW allows scope adjustment whilst ensuring core functionality delivery. Regular supervisor meetings provide early warning of issues.
Usability Evaluation Recruitment	Low	Participants recruited from honours cohort with adequate notice and flexible scheduling. Expert review can supplement user testing if needed.
Scope Creep	Medium	Requirements clearly documented and prioritised. Should and Could have requirements provide clear boundaries. Regular stakeholder communication manages expectations.
Data Security	Low	All data stored on secure HWU systems. Evaluation uses fictitious data. System follows established web security practices including input validation and secure database access.

prioritisation

Table 4. Project Risk Assessment

Overall, the project's risk profile is manageable, with most risks rated as low to medium severity. The use of proven technologies, early stakeholder engagement, and flexible requirements prioritisation provide a robust foundation for successful project completion.

C.3 PLES Issues

The development and evaluation of the Interactive Gantt Chart Builder addresses Professional, Legal, Ethical, and Social (PLES) considerations as follows, adhering to the British Computer Society (BCS) Code of Conduct [10].

Professional: The system implementation prioritises professional competence and integrity [10]. Development follows established software engineering principles and secure coding standards (e.g., OWASP guidelines [38]) to prevent vulnerabilities. The code is documented to industry standards to ensure maintainability for future HWU developers.

Legal: The project strictly complies with the UK Data Protection Act 2018 [24] and the GDPR [15]. All participant data is anonymised and stored on secure University servers. Furthermore, the project adheres to Intellectual Property (IP) rights; the software is an original creation, and any third-party libraries used are compliant with their respective open-source licences (e.g., MIT, Apache), ensuring no copyright infringement.

Ethical: In line with the University's ethical approval process, all evaluation participants will provide informed consent via information sheets, retaining the right to withdraw at any time without penalty. To mitigate harm, the system uses fictitious project data during testing, ensuring no student's actual academic work is at risk during the beta phase.

Social: The tool promotes digital inclusion by addressing the needs of students with varying levels of project management experience. To ensure equitable access, the interface aims to meet WCAG 2.1 AA accessibility standards [29], complying with the Equality Act 2010 [23], ensuring that students with visual or motor impairments can utilise the educational tool effectively.

D Summary of Changes

The conclusion was moved to section 7 and edited to flow better. Section 5 was changed to Implementation.

E Questionaire Results

User	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
1	4	1	5	1	4	1	5	1	5	3
2	4	1	4	2	4	2	4	1	4	2
3	5	1	5	1	5	1	5	1	5	1
4	5	1	5	2	5	2	5	1	4	1
5	5	1	5	4	5	2	5	1	4	2
6	5	1	5	1	5	1	5	1	5	1
7	5	1	5	1	5	1	5	1	5	2
8	5	2	5	1	5	2	4	1	4	2
9	5	3	4	3	5	2	4	3	4	4
10	5	1	5	1	5	1	5	1	5	1
11	5	2	4	2	5	2	4	1	4	1
12	5	1	5	1	5	1	5	1	4	1
13	5	3	4	2	4	2	4	2	4	4
14	4	2	4	2	4	2	4	2	4	2
15	5	1	5	1	5	1	5	1	5	1

Table 5. User Survey Responses (Q1-Q10)

Table listed as 1-5, 1 being strongly disagree and 5 being strongly agree. The mean SUS score calculated from these responses is 87.8, which corresponds to an adjective rating of Excellent on Bangor et al.'s scale [6].

User	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8
1	Very Easy	Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
2	Easy	Easy	Easy	Very Easy	Easy	Easy	Very Easy	Easy
3	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
4	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
5	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
6	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
7	Very Easy	Easy	Very Easy	Very Easy	Easy	Easy	Very Easy	Very Easy
8	Very Easy	Very Easy	Easy	Very Easy	Very Easy	Very Easy	Very Easy	Easy
9	Very Easy	Very Easy	Very Easy	Very Easy	Neutral	Very Easy	Very Easy	Very Easy
10	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
11	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
12	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy	Very Easy
13	Very Easy	Easy	Easy	Did not use	Easy	Very Easy	Very Easy	Very Easy
14	Very Easy	Very Easy	Easy	Very Easy	Easy	Very Easy	Very Easy	Very Easy
15	Very Easy	Very Easy	Very Easy	Very Easy	Easy	Very Easy	Very Easy	Very Easy

Table 6. User Ease-of-Use Ratings for Core Features

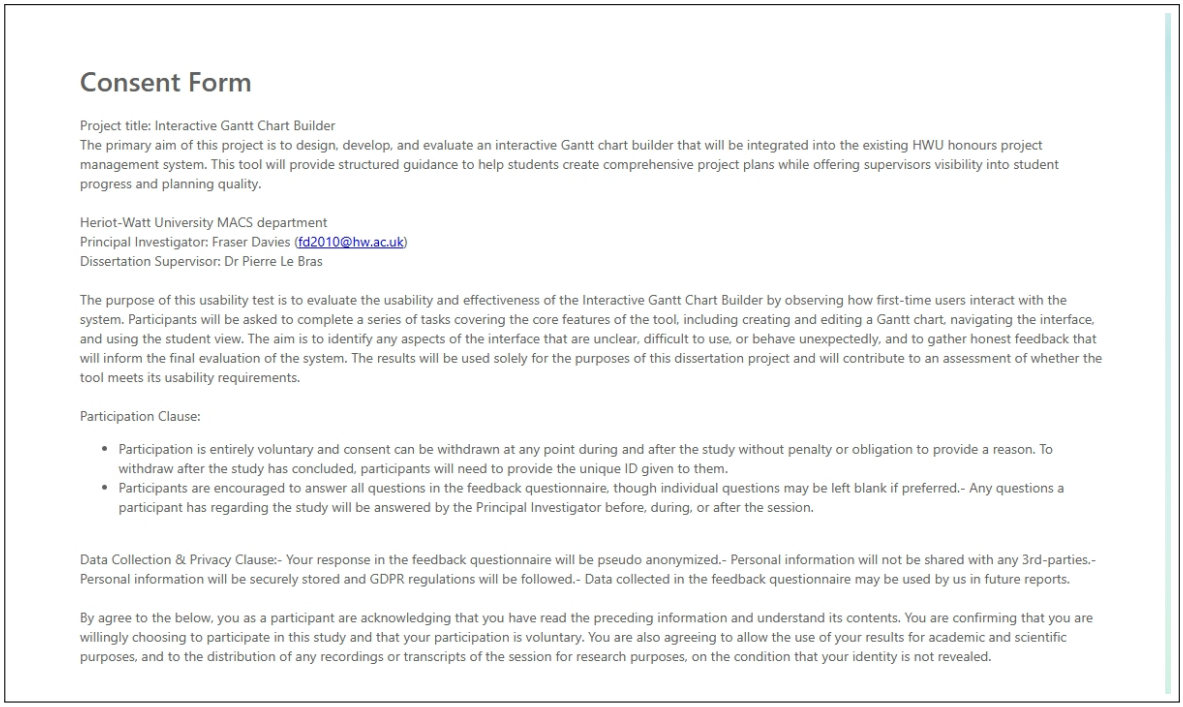
User	Easiest Part	Most Difficult Part
1	Opening the kanban board	Getting used to dragging
2	Dragging bars	Don't know
3	Kanban	Milestones are slightly hard
4	Creating new tasks	N/a
5	Dependencies were straight forward	The small bugs that still exist
6	Going through the tutorial	0
7	Making a task	Task progress
8	Creating the tasks	I think the dependence's
9	Creating the actual project	Using the progress bar
10	Creating tasks	Remembering to do a dependency
11	Adding in tasks	Shortening tasks
12	Making new tasks	Adding subtasks
13	Creating subtasks, the plans	Creating new individual tasks
14	Adding a task, and resizing	Adding a subtask
15	Dragging in tasks	Making a subtask

Table 7. User Feedback: Ease of Use and Challenges

User	Missing Anything?	Tutorial Helpfulness	Other Feedback
1	No	It did help	Make the colour choices...
2	No	I closed it but opened it	Nah
3	No	Yes it did help, I read it	Nope
4	No	Yes. It helped	No, it's all ready class
5	No	Yes, absolutely explained...	Just minor bug tweaks.
6	No	Tutorial was great and very...	No
7	Nah	Absolutely. A clear and...	I would add the ability to...
8	Nope	It was helpful	Dependence's, since we...
9	No really no	Yes it did, it gave me...	Nah.
10	No	Yes it was great	No
11	No	Yes it great	No
12	No	0	0
13	Change colour for different...	Yes, the tutorial was bril...	Group edit button with the...
14	Assign roles to people	Helped a lot	Move the edit to the top
15	Nah	Tutorial helped understand...	Do more with subtasks

Table 8. User Feedback: Tutorial and Suggestions

F Consent Form



Consent Form

Project title: Interactive Gantt Chart Builder
The primary aim of this project is to design, develop, and evaluate an interactive Gantt chart builder that will be integrated into the existing HWU honours project management system. This tool will provide structured guidance to help students create comprehensive project plans while offering supervisors visibility into student progress and planning quality.

Heriot-Watt University MACS department
Principal Investigator: Fraser Davies (fd2010@hw.ac.uk)
Dissertation Supervisor: Dr Pierre Le Bras

The purpose of this usability test is to evaluate the usability and effectiveness of the Interactive Gantt Chart Builder by observing how first-time users interact with the system. Participants will be asked to complete a series of tasks covering the core features of the tool, including creating and editing a Gantt chart, navigating the interface, and using the student view. The aim is to identify any aspects of the interface that are unclear, difficult to use, or behave unexpectedly, and to gather honest feedback that will inform the final evaluation of the system. The results will be used solely for the purposes of this dissertation project and will contribute to an assessment of whether the tool meets its usability requirements.

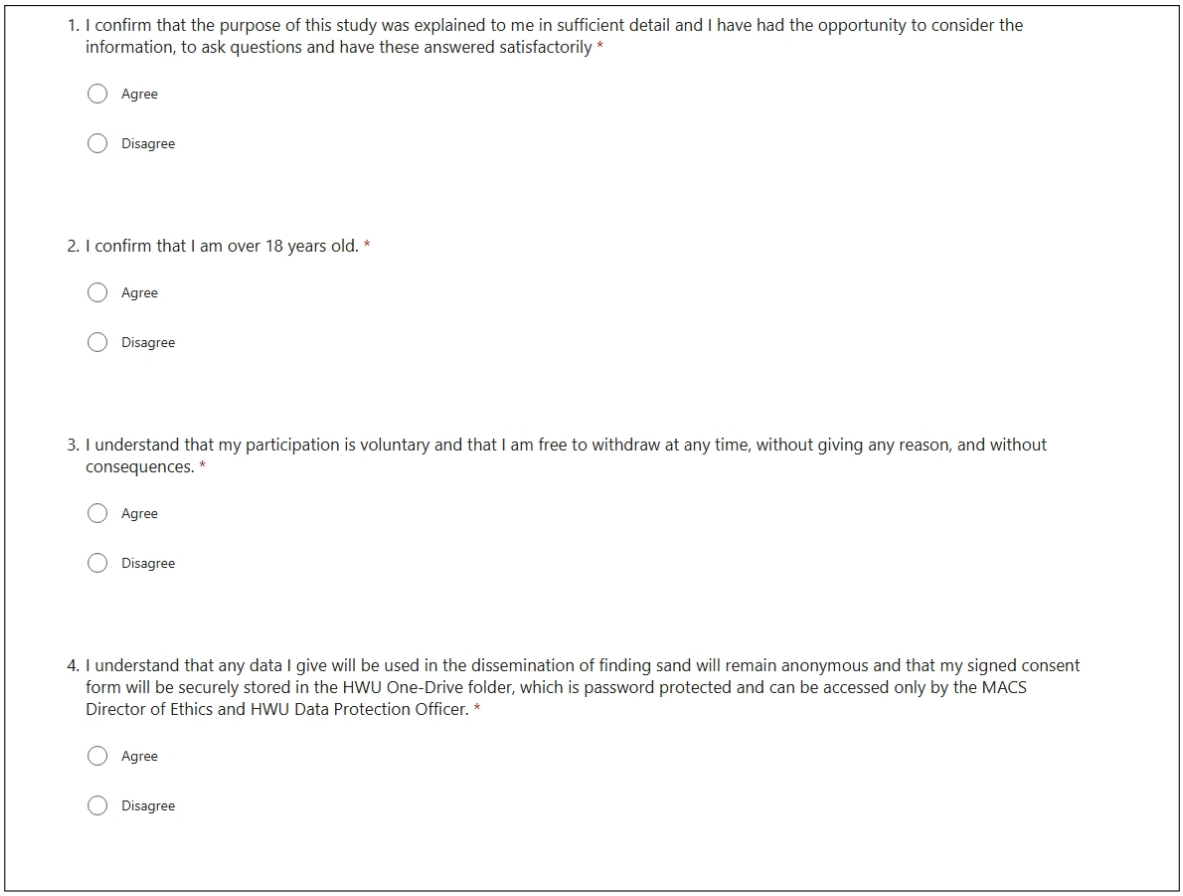
Participation Clause:

- Participation is entirely voluntary and consent can be withdrawn at any point during and after the study without penalty or obligation to provide a reason. To withdraw after the study has concluded, participants will need to provide the unique ID given to them.
- Participants are encouraged to answer all questions in the feedback questionnaire, though individual questions may be left blank if preferred.- Any questions a participant has regarding the study will be answered by the Principal Investigator before, during, or after the session.

Data Collection & Privacy Clause:- Your response in the feedback questionnaire will be pseudo anonymized.- Personal information will not be shared with any 3rd-parties.- Personal information will be securely stored and GDPR regulations will be followed.- Data collected in the feedback questionnaire may be used by us in future reports.

By agree to the below, you as a participant are acknowledging that you have read the preceding information and understand its contents. You are confirming that you are willingly choosing to participate in this study and that your participation is voluntary. You are also agreeing to allow the use of your results for academic and scientific purposes, and to the distribution of any recordings or transcripts of the session for research purposes, on the condition that your identity is not revealed.

Fig. 10. Consent Form



1. I confirm that the purpose of this study was explained to me in sufficient detail and I have had the opportunity to consider the information, to ask questions and have these answered satisfactorily *

- Agree
- Disagree

2. I confirm that I am over 18 years old. *

- Agree
- Disagree

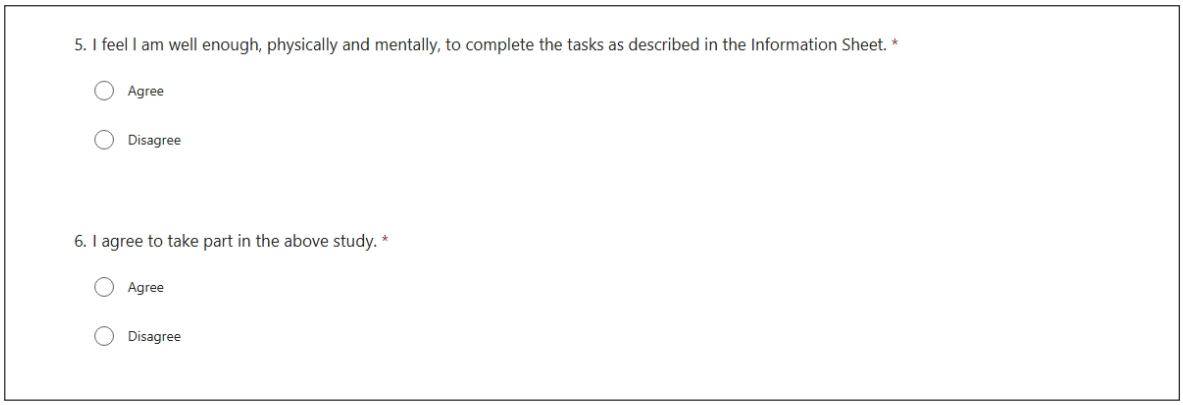
3. I understand that my participation is voluntary and that I am free to withdraw at any time, without giving any reason, and without consequences. *

- Agree
- Disagree

4. I understand that any data I give will be used in the dissemination of finding sand will remain anonymous and that my signed consent form will be securely stored in the HWU One-Drive folder, which is password protected and can be accessed only by the MACS Director of Ethics and HWU Data Protection Officer. *

- Agree
- Disagree

Fig. 11. Consent Form



5. I feel I am well enough, physically and mentally, to complete the tasks as described in the Information Sheet. *

- Agree
- Disagree

6. I agree to take part in the above study. *

- Agree
- Disagree

Fig. 12. Consent Form

G Generative AI

Generative artificial intelligence tools were used during the preparation of this dissertation to assist with literature search and reference discovery, and to support writing quality through spelling, grammar, punctuation, and sentence structure improvements. Generative AI was also used in troubleshooting isues with text editors such as LaTex in the creation of this document. All references identified through AI assistance were systematically verified by hand to ensure legitimacy, relevance, and that cited content accurately supported the claims made. Generative AI functioned solely as a supportive tool rather than a generator of primary content or ideas.

During the implementation phase, generative AI tools were used in a number of supporting capacities. AI assistance was used to generate realistic dummy data for seeding the database during development and testing, and to reformat code, particularly CSS stylesheets, to improve consistency and readability. AI tools were also used to help identify and fix errors during debugging, and as a sounding board for discussing potential solutions to technical problems encountered throughout development. Throughout the implementation phase, AI was used extensively as a spelling and grammar checker when writing code comments and documentation. In all cases, the underlying logic, architectural decisions, and implementation choices remained the work of the author, with AI serving only as a supportive aid.